TECHNICAL UNIVERSITY OF KENYA

FACULTY OF APPLIED SCIENCES AND TECHNOLOGY

SCHOOL OF COMPUTING AND INFORMATION TECHNOLOGY

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATICS

BACHELOR OF TECHNOLOGY (INFORMATION TECHNOLOGY)

PROJECT TITLE:

DEVELOPMENT OF A WEB-BASED LABORATORY INFORMATION MANAGEMENT SYSTEM (LIMS): A CASE OF YALA SUB-COUNTY HOSPITAL

PRESENTED BY:

OWUOR COLLINS

SCCI/01227/2022

SUPERVISED BY:

Dr. Edwin Ngwawe &

Mr. Peter Maina Kariuki

PROJECT SUBMITTED TO THE SCHOOL OF COMPUTING AND INFORMATION TECHNOLOGY IN PARTIAL FULFILMENT FOR THE BACHELOR OF TECHNOLOGY IN INFORMATION TECHNOLOGY OF THE TECHNICAL UNIVERSITY OF KENYA

SUBMISSION DATE: APRIL 2026

DECLARATION

I hereby declare that this is my original work and has not been presented previously in another institution for any academic certificate, diploma or degree.

OWUOR COLLINS
SCCI/01227/2022

Signature: ..... Date: .....

SUPERVISORS

Dr. Edwin Ngwawe
The Technical University of Kenya
Signature: ..... Date: .....

Mr. Peter Maina Kariuki
The Technical University of Kenya
Signature: ..... Date: .....

DEDICATION

I dedicate this work to my family for their unwavering financial, moral, and motivational support throughout every stage of this project. To my supervisors, Dr. Edwin Ngwawe and Mr. Peter Maina Kariuki, whose guidance shaped this work. To the staff of Yala Sub-County Hospital for their cooperation. And to the Almighty God for His divine protection and provision of good health and knowledge throughout this journey.

ACKNOWLEDGEMENT

First and foremost, I would like to thank the Almighty God for great health, wisdom, and knowledge throughout the duration of this project.

I sincerely thank my supervisors, Dr. Edwin Ngwawe and Mr. Peter Maina Kariuki, for their continuous support, constructive critique, and guidance during the development and documentation of this project.

My gratitude goes to the management and staff of Yala Sub-County Hospital — particularly the Laboratory In-Charge and Medical Superintendent — for granting access to the facility and cooperating during data collection, system analysis, and User Acceptance Testing.

Lastly, I thank my family and classmates for their invaluable support throughout the research and development period.

ABSTRACT

Medical record management at sub-county level public health facilities in Kenya remains heavily paper-based, resulting in significant inefficiencies in laboratory service delivery. At Yala Sub-County Hospital, the existing manual workflow — characterised by handwritten MOH 204 request forms and MOH 240 laboratory registers — causes turnaround time delays, transcription errors, lost result slips, and poor data traceability, all of which directly impair patient care.

This project presents the design, development, and deployment of a web-based Laboratory Information Management System (LIMS) for Yala Sub-County Hospital. The system was built using PHP (procedural architecture), MySQL as the relational database engine, and Bootstrap 5 for a responsive frontend interface, deployed over the hospital's Local Area Network (LAN) via XAMPP.

The system implements comprehensive core operational modules: Patient Registration with unique OPD numbering and A5 printable routing slips featuring Code128 barcodes, Electronic Test Ordering (CPOE) with clinical allergy warnings, Laboratory Testing Workbench with automated reference range validation and critical panic value alerting, Inpatient Blood Bank crossmatching, Specimen Rejection QA (ISO 15189), Cashless Revenue Collection via Kenya Government eCitizen (Paybill 222222) with dual-emblem A5 receipting, and an Administrative Dashboard with automated MOH 706 statistical reporting. Security features include bcrypt password hashing, CSRF token protection, MySQLi prepared statements for SQL injection prevention, strict Role-Based Access Control (RBAC) across four user roles, and an immutable audit trail.

User Acceptance Testing (UAT) was conducted with 8 hospital staff members representing all user roles. The overall average satisfaction score was 4.6 out of 5.0, with all participants expressing a clear preference for the digital system over the existing manual process. The project demonstrates that a cost-effective open-source web system can successfully bridge the digital divide in resource-constrained public health facilities.

Keywords: Laboratory Information Management System, LIMS, PHP, MySQL, Role-Based Access Control, Yala Sub-County Hospital, Kenya, M-Pesa, Health Informatics.

LIST OF FIGURES

Figure 1: Project Gantt Chart 15
Figure 2: Budget and Resources..... 17
Figure 3: Agile Methodology Diagram 24
Figure 4: Flowchart of Existing Manual System 38
Figure 5: Flowchart of Proposed System 40
Figure 6: Use Case Diagram 41
Figure 7: Level 0 Data Flow Diagram..... 42
Figure 8: Level 1 Data Flow Diagram..... 43
Figure 9: Entity Relationship Diagram..... 46
Figure 10: phpMyAdmin Database Schema.....50
Figure 11: Login / Sign In Screen.....54
Figure 12: Dashboard — KPI Statistics 55
Figure 13: Dashboard — Module Navigation..... 56
Figure 14: Register New Patient Screen..... 57
Figure 15: Register Patient — Insurance Section..... 58
Figure 16: Test Ordering / Doctor's Console.....59
Figure 17: Lab Workbench — Lab Queue 60
Figure 18: Results Entry Screen — Paid Request 61
Figure 19: Results Entry Screen — Unpaid Request..... 62
Figure 20: Patient History Screen..... 63
Figure 21: Patient History — Visit Details 64
Figure 22: Patient History — Completed Requests..... 65
Figure 23: Reports & Analytics Dashboard.....66
Figure 24: Reports — Daily Trend Chart and Top Tests 67
Figure 25: Reports — Recent Transactions 68
Figure 26: Manage Users Screen 69
Figure 27: Test Catalogue Screen..... 70
Figure 28: Audit Log Screen 71
Figure 29: Bulk Import Patients Screen..... 72
Figure 30: Database Backup Screen..... 73
Figure 31: Cashier / Billing Screen (All Bills Settled)..... 74
Figure 32: Three-Tier System Architecture 26
Figure 33: Hospital LAN Deployment Topology 23

LIST OF TABLES

Table 1: Budget and Resources 17
Table 2: Hardware and Software Requirements 28
Table 3: Data Dictionary — Patients Table (Key Fields) 47
Table 3b: Payments Table Schema (Actual) 47
Table 4: System Roles and Permissions Matrix 48
Table 5: Test Cases and Results 70
Table 6: UAT Feedback Summary 73

CHAPTER 1: INTRODUCTION

1.1 Introduction

The modernisation of healthcare infrastructure is a central pillar of Kenya's Vision 2030 and the National Digital Health Strategy. While investments have been made in digitising records at national referral hospitals, the digital divide between national and sub-county level facilities remains significant. Yala Sub-County Hospital operates its laboratory entirely on manual paper-based workflows, resulting in a cascade of inefficiencies that directly compromise the quality and timeliness of patient care. This project addresses this gap by developing a localised, cost-effective, web-based Laboratory Information Management System (LIMS) specifically tailored for the hospital's operational context.

1.2 Background of the Study

Yala Sub-County Hospital is a Level 4 public health facility in Gem Sub-County, Siaya County, Western Kenya. As a primary referral centre for the region, it handles a high volume of outpatients and inpatients daily, offering laboratory services from basic haematology to viral load testing. Despite its critical role, the laboratory department operates entirely manually — relying on handwritten MOH 204 request forms, MOH 240 laboratory registers, and physical result slips carried between departments by patients or messengers. This system is slow, error-prone, and impossible to audit effectively.

1.3 Problem Statement

The reliance on paper-based registers at Yala Sub-County Hospital results in significant laboratory turnaround time delays, as staff spend disproportionate time transcribing data rather than analysing samples. The manual system lacks a reliable audit trail, making it difficult to establish who authorised a result or when a sample was collected. Transcription errors from illegible handwriting on MOH forms are a frequent source of patient safety incidents. The loss of physical result slips affects approximately 12% of all requests per month. Monthly MOH report compilation takes an average of two full working days due to manual tallying. There is a critical need for a digital solution that automates these workflows while remaining affordable and sustainable for a public hospital.

1.4 Objectives

1.4.1 General Objective

To design, develop, and evaluate a web-based Laboratory Information Management System (LIMS) for Yala Sub-County Hospital to automate diagnostic workflows, ensure data accuracy, and improve clinical turnaround times.

1.4.2 Specific Objectives

- 1. To assess and model user requirements and manual workflow bottlenecks across outpatient, clinical consultation, and laboratory testing departments at Yala Sub-County Hospital.
- 2. To design the 3-tier system architecture, relational database schema, and role-based access interfaces (RBAC) for Receptionist, Medical Doctor, Laboratory Technologist, Cashier, and Administrator.
- 3. To develop and implement the core diagnostic modules: MOH 204 outpatient routing with Code128 barcodes, CPOE test ordering with allergy flags, laboratory results entry with automated reference ranges and panic alerting, inpatient blood bank safety, specimen rejection QA (ISO 15189), and cashless revenue collection via Kenya Government eCitizen

(Paybill 222222).

- 4. To test, validate, and evaluate the system's performance, security, and automated MOH 706 statistical reporting through User Acceptance Testing (UAT) with hospital staff.

-

1.5 Justification

The successful implementation of this project will benefit all stakeholders: the hospital gains reduced stationery costs and improved billing accuracy; clinicians gain instant access to patient test history; patients benefit from reduced waiting times and no lost results; administrators receive automated MOH-compliant reports; and the Ministry of Health gains more accurate disease surveillance data from Yala Sub-County.

1.6 Scope of the Study

In Scope: Design and development of a 3-tier web architecture on PHP 8.2 and MySQL 8.0; responsive Bootstrap 5 user interfaces with strict Role-Based Access Control; MOH 204 outpatient routing slips with Code128 barcodes; CPOE diagnostic ordering; laboratory workbench with automated reference range checking and panic alerting; inpatient blood bank crossmatching; specimen rejection QA (ISO 15189); cashless eCitizen Paybill 222222 payment gateway integration; automated FPDF PDF certification; and deployment on the hospital's intranet Local Area Network (LAN) via Apache/XAMPP. Out of Scope: Direct bidirectional serial interfacing (RS-232/HL7) with physical laboratory machines, live national cloud synchronization with DHIS2/KHIS servers, and physical hospital network cabling.

1.7 Limitations of the Proposed System

- Infrastructure dependency on a stable LAN and power supply — mitigated by UPS recommendations.
- English-only interface — may present a barrier to staff with limited English literacy.
- Long-term maintenance requires local IT support after initial deployment.

1.8 Project Risk and Mitigation

- System failure: Automated daily database backups and manual backup trigger for the Administrator.
- Unauthorised access: RBAC, bcrypt passwords, and CSRF protection implemented throughout.
- Staff resistance: Interface designed to mirror familiar manual workflow; early UAT engagement.
- Data loss during development: All source code version-controlled using GitHub.

1.9 Project Schedule

The project was executed over six months using the Agile methodology, from conceptualisation through deployment. The schedule is shown in the Gantt chart below.

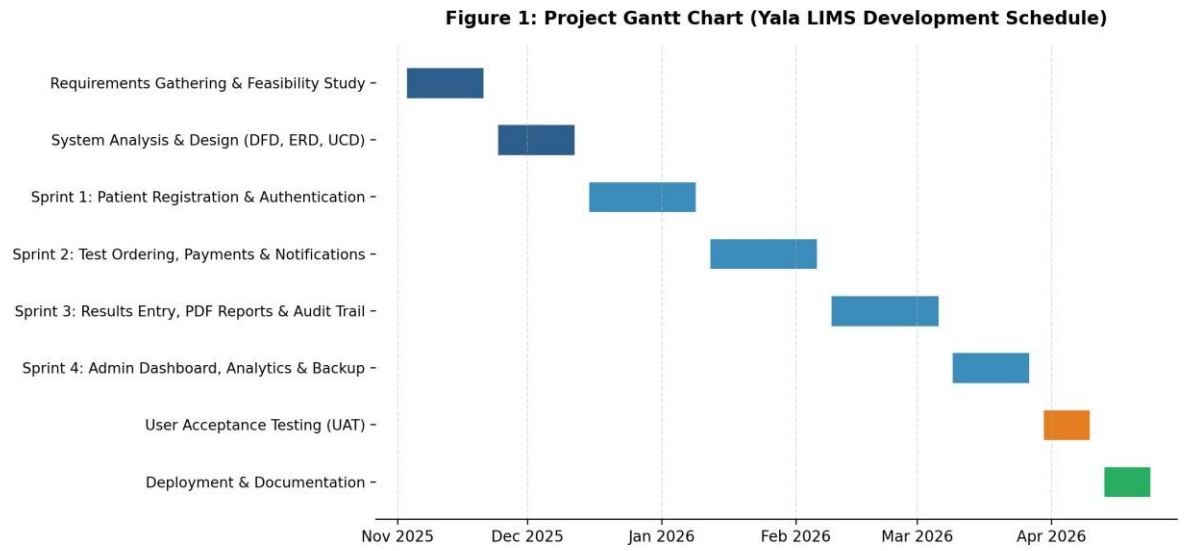


Figure 1: Project Gantt Chart

1.10 Budget and Resources

Item	Amount (KES)	Description
Laptop (Core i5, 8GB RAM)	45,000	Primary coding and deployment machine
Mobile Phone (Android)	20,000	For testing system responsiveness
Internet Access (6 months)	12,000	Research, GitHub, cloud backups
Cloud / Shared Hosting	5,000	Optional external hosting
Cloud Backup Storage	3,000	Minimum 15 GB
Antivirus Software	2,000	Kaspersky — 1-year licence
Transport & Logistics	10,000	Field visits to Yala Sub-County Hospital
Stationery & Printing	6,000	Binding of proposal and final report
Contingency	7,800	Unforeseen expenses
TOTAL	KES 110,800	

Table 1: Budget and Resources

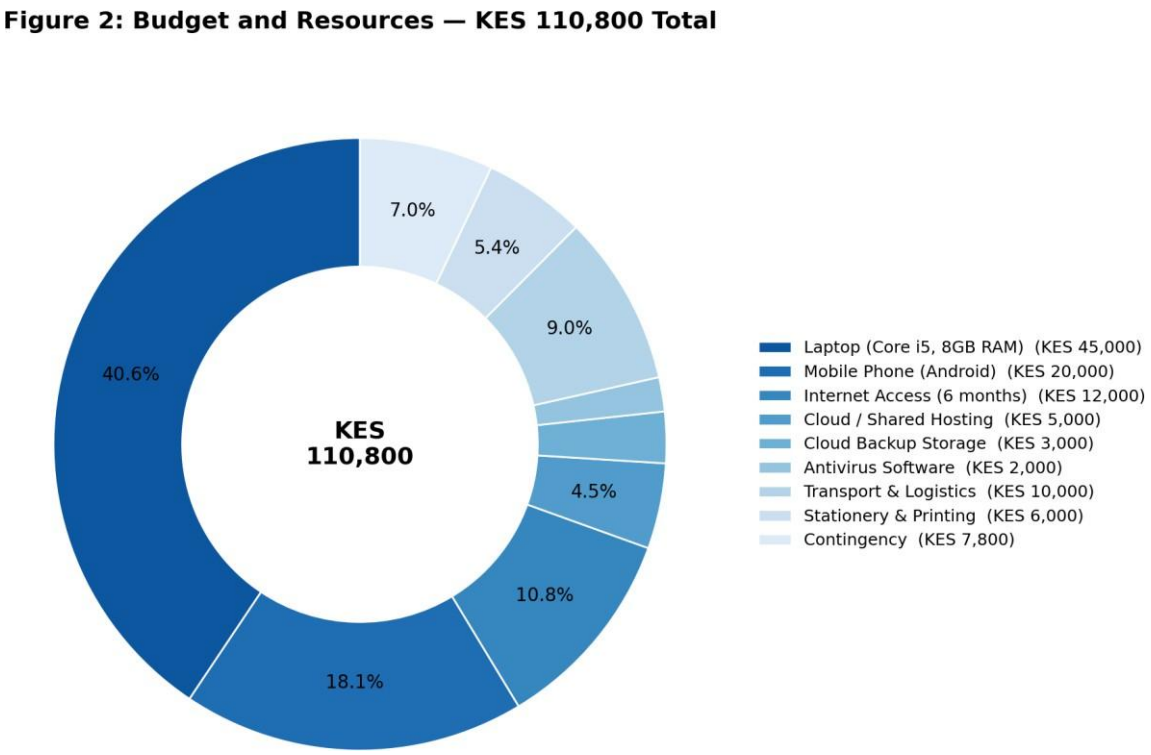


Figure 2: Budget and Resources — Proportional Breakdown

CHAPTER 2: LITERATURE REVIEW

The concept of digitising medical records to improve healthcare delivery is a subject of global interest. This chapter analyses existing systems — from the traditional manual methods at Yala Sub-County Hospital to advanced electronic platforms — to identify the gaps that the proposed system addresses.

2.1 Reviewed Similar Systems

2.1.1 The Manual Paper-Based System (MOH Standard)

This represents the current standard at the majority of Level 3 and 4 public facilities in Kenya (MOH, 2020). It relies on MOH 204 request forms and MOH 240 laboratory registers. Monthly reports are generated by manually tallying register entries. The system has been in place for decades due to its low barrier to entry and zero infrastructure requirement.

2.1.2 OpenMRS (Open Medical Record System)

An open-source EMR widely used in Kenya for HIV/AIDS programmes (AMPATH in Western Kenya). It supports longitudinal patient tracking and international data coding standards. However, its resource requirements and complexity make it unsuitable for Level 4 facilities with limited IT infrastructure.

2.1.3 Commercial LIMS (Proprietary Systems)

Systems such as LabWare LIMS, Thermo Fisher SampleManager, STARLIMS, and LabVantage provide comprehensive automation and direct analyser interfacing. Their licensing costs — often millions of shillings — make them unaffordable for county-funded public hospitals (Zahra & Patel, 2018).

2.2 Tools and Methodologies Used in Reviewed Systems

The manual system uses a transcription methodology with no logical validation. OpenMRS uses a Java/React enterprise architecture. Commercial LIMS use proprietary closed stacks. All three represent different points on the cost-complexity spectrum, leaving a gap specifically for lightweight, open-source, affordable solutions tailored to sub-county level facilities.

2.3 Gaps in Existing Systems and Proposed Solution

The manual system's gap is efficiency and accuracy. OpenMRS is too heavy and complex for Level 4 facilities. Commercial LIMS are unaffordable. The Yala LIMS addresses all three gaps: it is lightweight (runs on XAMPP over LAN), open-source (no licensing costs), and purpose-built for the specific workflows of Yala Sub-County Hospital's laboratory department.

CHAPTER 3: METHODOLOGY

3.1 Methodology and Tools

3.1.1 Methodology

The Agile Methodology was adopted for this project. Unlike the rigid Waterfall model, Agile offers an iterative and incremental approach that is well-suited to a context where requirements are discovered through close engagement with end users and where mid-development changes — such as the addition of insurance payment support — are necessary.

Figure 3: Agile Methodology — Iterative Sprint Cycle Used in Yala LIMS Development

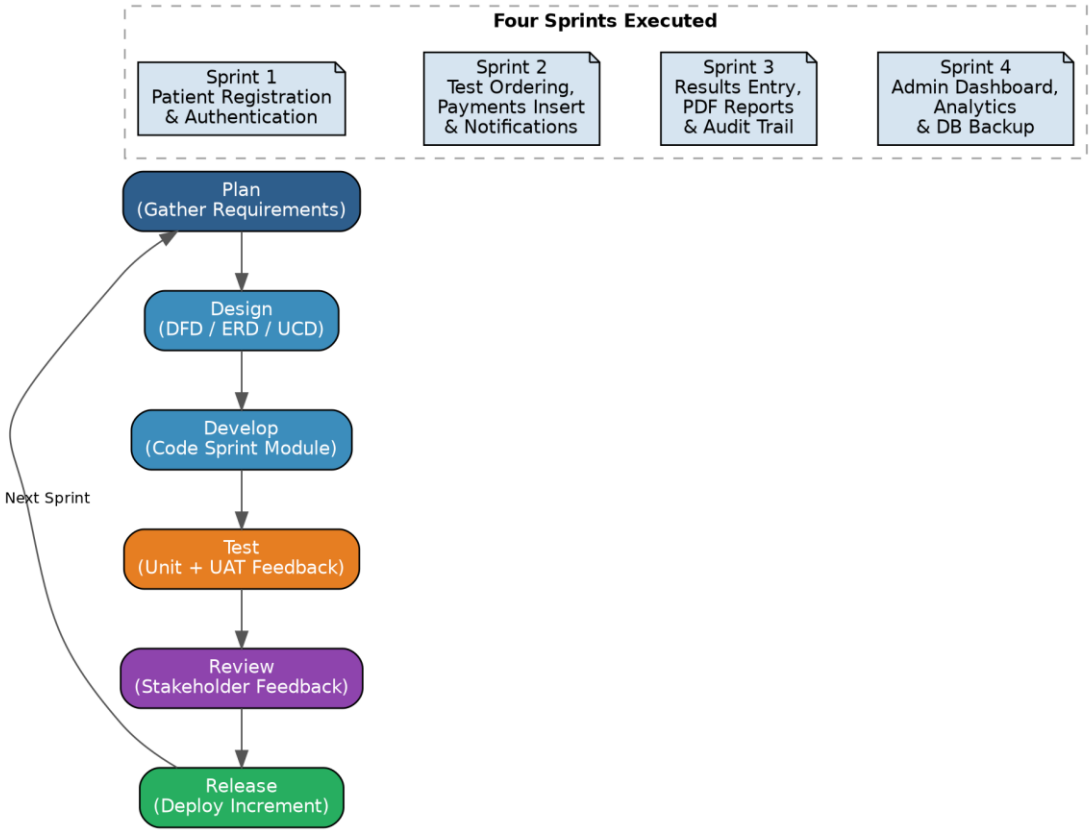


Figure 3: Agile Methodology

3.1.1.1 Justification for Agile

- Accommodates changing requirements at any stage without derailing the project.
- Delivers working modules early — each sprint produced a testable component for UAT feedback.
- Risk is managed in small iterations — a failing module can be fixed without breaking completed ones.
- Mirrors hospital staff workflows closely through continuous feedback integration.

3.1.1.2 Agile Sprints

- Sprint 1: Patient Registration and Authentication Module
- Sprint 2: Test Ordering, Payments Row Insert, and Notifications Module
- Sprint 3: Results Entry, PDF Report Generation, and Audit Trail Module
- Sprint 4: Administrative Dashboard, Reports Analytics, and Database Backup Module

3.1.2 System Design Tools

- Flowcharts: Sequential process flow for each module — drawn in Lucidchart.
- Use Case Diagrams: User-system interaction for all four roles — drawn in Visual Paradigm Online.
- Data Flow Diagrams (DFD): Level 0 and Level 1 — drawn in Lucidchart.
- Entity-Relationship Diagram (ERD): Relational database schema — drawn in dbdiagram.io.

3.2 Source of Data

Primary data was collected through field observation visits at Yala Sub-County Hospital, key informant interviews with the Lab In-Charge and Medical Superintendent, and structured questionnaires administered to 10 staff members. Secondary data was drawn from MOH standard operating procedures, open-source LIMS documentation, and academic literature on health informatics.

3.3 Data Collection Methods

3.3.1 Observation

A fly-on-the-wall approach was used during two field visits. Specific metrics recorded included average time per paper request, frequency of lost request forms, and daily test volume per technician.

3.3.2 Questionnaires

Structured questionnaires with closed and open-ended questions provided anonymity for honest staff feedback on the manual system's shortcomings.

3.3.3 Interviews

One-on-one interviews with the Lab In-Charge and Medical Superintendent provided detailed functional requirements that could not be captured by questionnaire alone.

3.4 Resources Required

Category	Resource	Spec	Unit KES	Total KES
Hardware	Development Laptop	Core i5, 8GB, 500GB SSD	90,000	90,000
	Android Mobile Device	Standard Android	20,000	20,000
	UPS / Power Backup	650VA	8,000	8,000
Software	Windows 10/11	64-bit	18,000	18,000
	VS Code, XAMPP, MySQL, GitHub	Free / Open source	0	0
	Kaspersky Antivirus	1-Year	3,000	3,000
Internet	Internet Subscription	4 Months	4,000/mo	16,000
TOTAL				155,000

Table 2: Hardware and Software Requirements

CHAPTER 4: SYSTEM ANALYSIS AND REQUIREMENT MODELLING

4.1 Introduction

This chapter presents the structural analysis of the existing manual system at Yala Sub-County Hospital and documents the detailed requirements gathered to guide the design of the proposed LIMS. It covers the feasibility study, functional and non-functional requirements, and the system design tools used to model the application's processes and data structures.

4.2 Feasibility Study

4.2.1 Technical Feasibility

The hospital has an existing Local Area Network (LAN) connecting reception, consultation rooms, and the laboratory. Workstation computers are available at reception and laboratory stations. The designated server machine meets the minimum specifications for XAMPP. No additional hardware investment is required for basic deployment. The diagram below shows how the LIMS server, network switch, and role-based workstations are physically deployed across the hospital LAN, with UPS backup protecting the server against power interruptions.

Figure 33: Hospital LAN Deployment Topology — Yala Sub-County Hospital

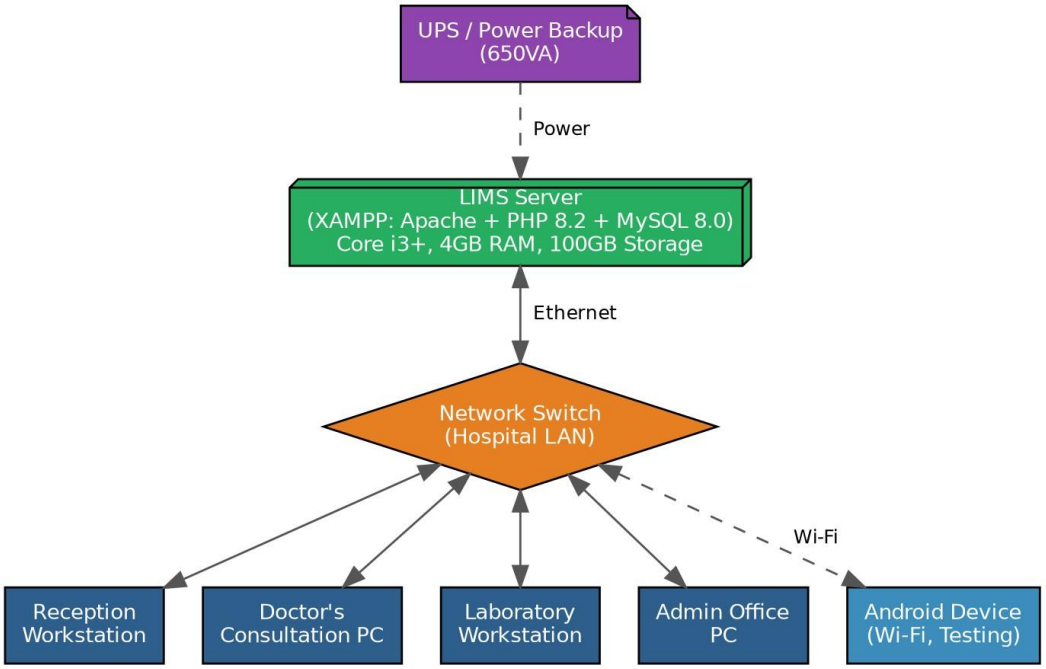


Figure 33: Hospital LAN Deployment Topology

4.2.2 Economic Feasibility

The total development cost of KES 110,800 compares favourably against the annual stationery cost and the labour cost of manual monthly MOH report compilation. The improved billing accuracy from the digital payments module is expected to recover previously un-billed laboratory services.

4.2.3 Operational Feasibility

The proposed system does not alter the fundamental workflow — test ordering, sample processing, result reporting, and billing remain the same sequential steps. The interface is designed to mirror the familiar paper-based workflow. UAT results confirmed all staff rated ease of use at 4.0 or above out of 5.0.

4.2.4 Schedule Feasibility

The project was completed within the academic calendar period through four Agile sprints across the development semester.

4.3 System Requirements

4.3.1 Functional Requirements

Patient Registration: Allow registration with unique OPD number, full name, demographics, and optional insurance details. Support bulk CSV import for existing records.

Test Ordering (CPOE): Allow Doctors to electronically select tests from a categorised catalogue, automatically calculate total cost, create a lab request, and notify the Lab Technician.

Lab Queue & Sample Tracking: Display pending/in-progress requests with status badges (Pending, In Progress, Completed, Overdue, Unpaid).

Results Entry: Allow Lab Technicians to enter result values and remarks per test, save results, update request status to Completed, and notify the ordering Doctor.

Billing & Payments: Allow Receptionists to record payments via M-Pesa, Cash, Insurance, or Waiver. Support insurance claim details and patient co-pay recording.

Administrative Dashboard: Display KPI cards, 7-day trend charts, top tests ranking, doctor activity, and recent transactions.

Audit Trail: Log all system actions with user identity, action type, affected record, and timestamp.

4.3.2 Non-Functional Requirements

Usability: Responsive Bootstrap 5 interface accessible on desktop and mobile browsers.

Performance: All pages load within 3 seconds on the hospital LAN.

Security: Bcrypt passwords, CSRF protection, MySQLi prepared statements, RBAC.

Reliability: Automated daily database backups; manual backup trigger for Admin.

Availability: System available 24/7 on the hospital LAN without internet dependency.

4.4 System Analysis Tools

4.4.1 Flowchart of Existing System

The flowchart below illustrates the current three-stage manual workflow: paper MOH 204 form → MOH 240 lab register transcription → physical result slip return to consultation room.

Figure 4: Flowchart of the Existing Manual System at Yala Sub-County Hospital

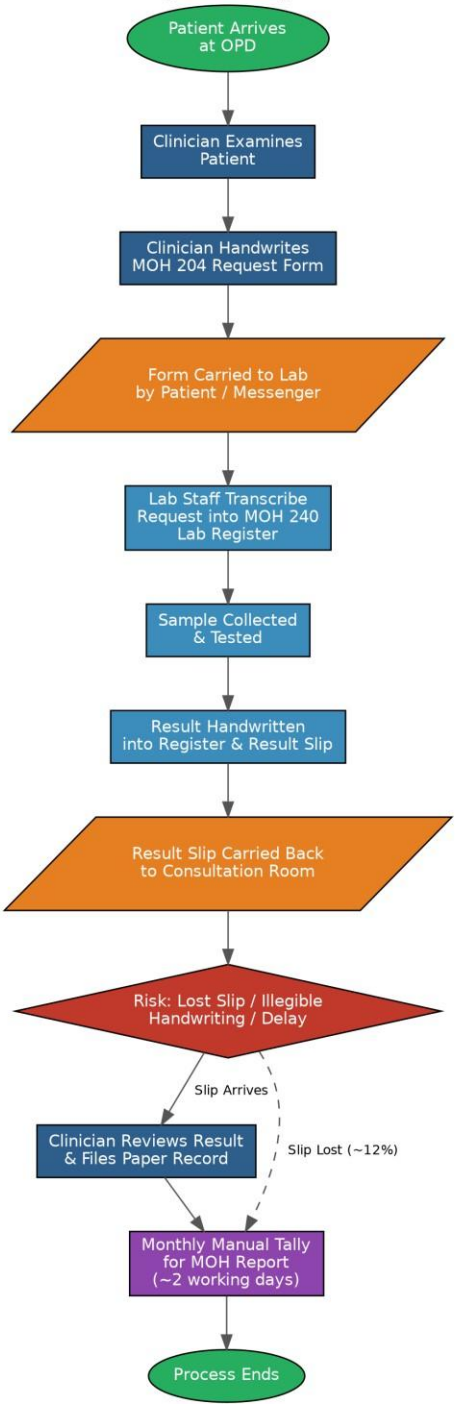


Figure 4: Flowchart of Existing Manual System

CHAPTER 5: SYSTEM DESIGN

5.1 Introduction

This chapter describes the complete design of the Yala LIMS, including the proposed system flowchart, use case diagram, data flow diagrams, entity-relationship diagram, data dictionary, database schema, and screen designs. The system follows a three-tier client-server architecture: presentation layer (Bootstrap 5), application layer (PHP 8.2 on Apache via XAMPP), and data layer (MySQL 8.0).

Figure 32: Three-Tier System Architecture — Yala LIMS

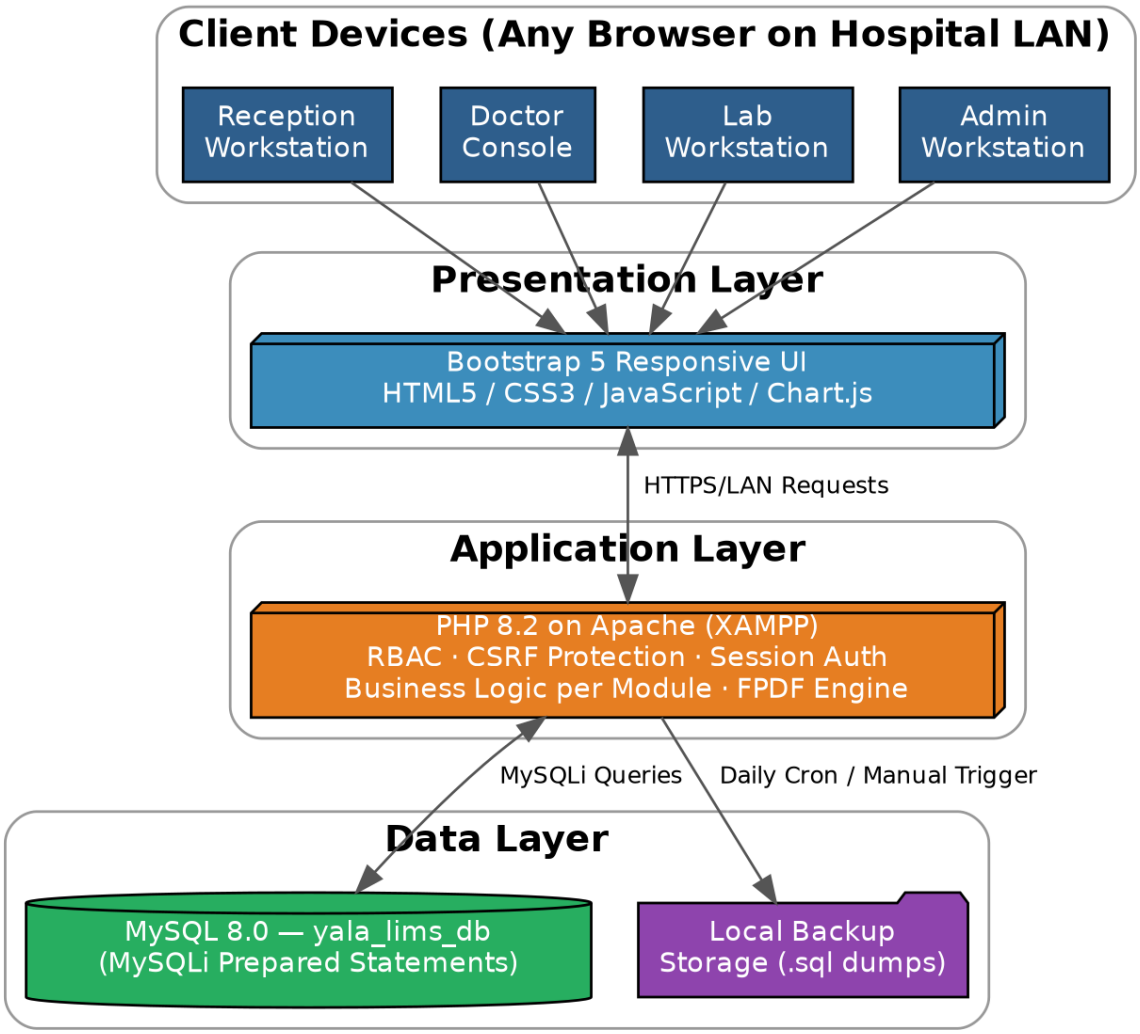


Figure 32: Three-Tier System Architecture

5.2 Program Design Tools

5.2.1 Flowchart of Proposed System

The proposed system flowchart shows the sequential digital workflow from login through patient registration, test ordering, lab processing, results entry, billing, and report generation.

Figure 5: Flowchart of the Proposed Yala LIMS Digital Workflow

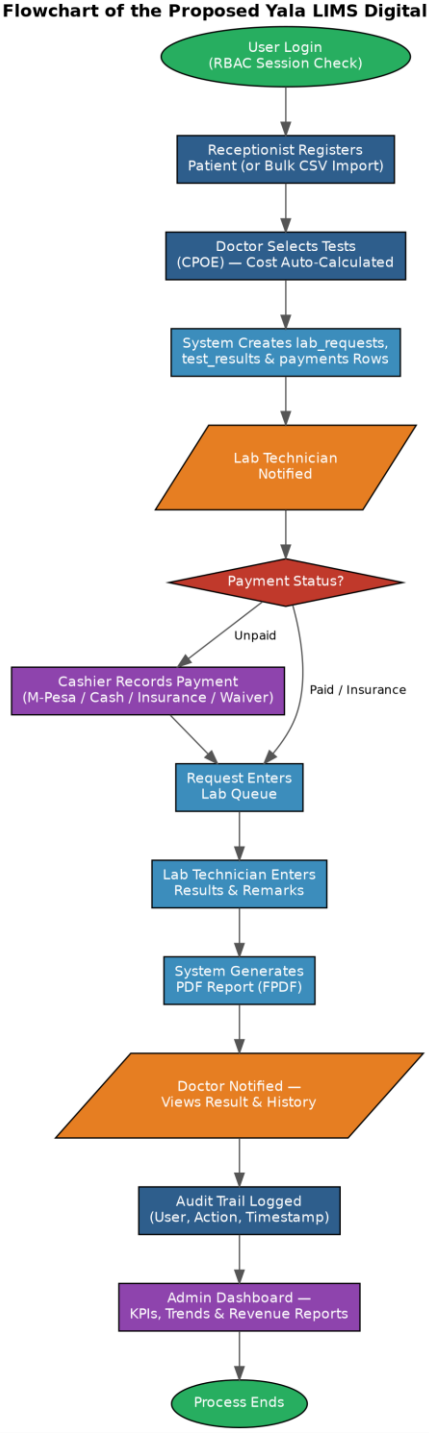


Figure 5: Flowchart of Proposed System

5.2.2 Use Case Diagram

The use case diagram shows interactions between four system actors — Receptionist, Doctor, Lab Technician, and Administrator — and the system modules available to each based on their RBAC role.

Figure 6: Use Case Diagram — Yala LIMS

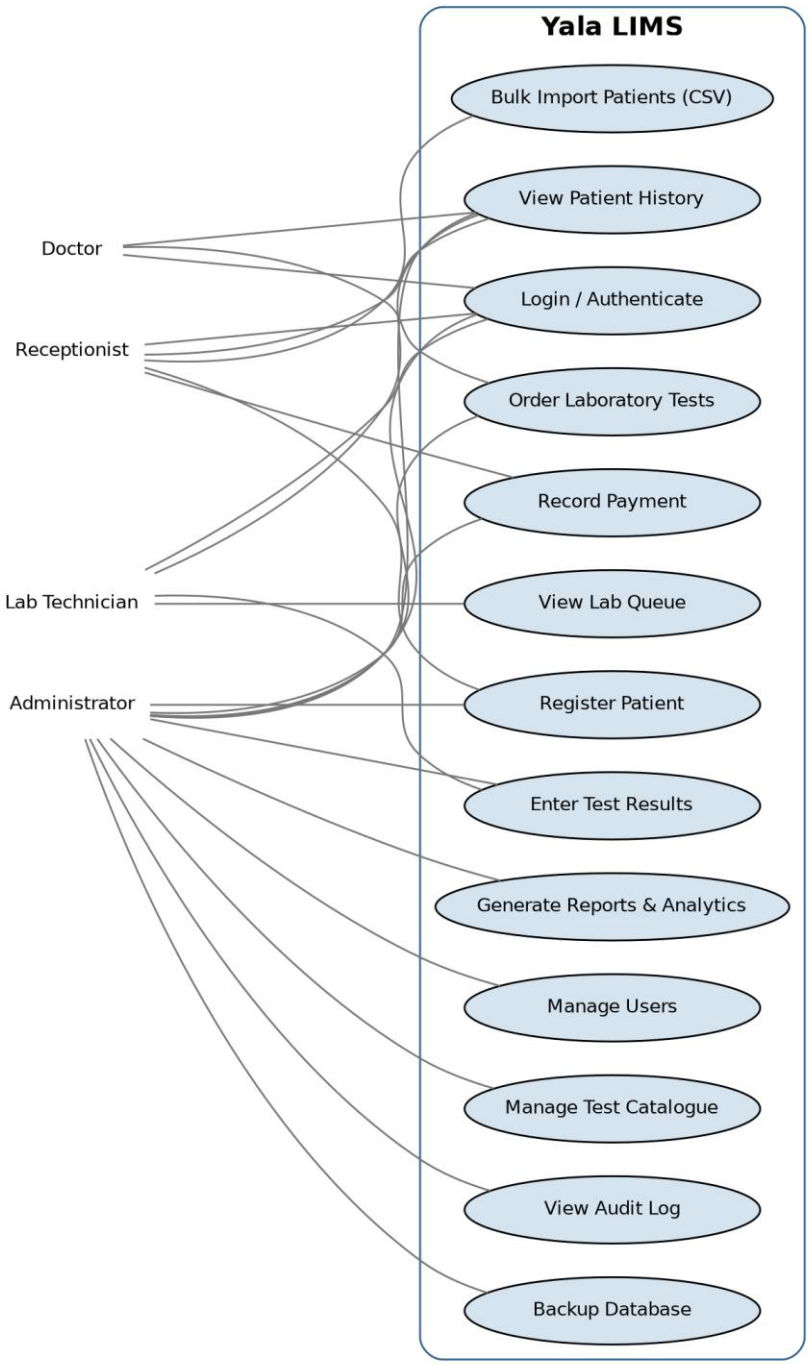


Figure 6: Use Case Diagram

5.2.3 Data Flow Diagram — Level 0 (Context Diagram)

The Level 0 DFD shows the Yala LIMS as a single process with four external entities: Doctor, Lab Technician, Receptionist/Cashier, and System Administrator.

Figure 7: Level 0 Data Flow Diagram (Context Diagram) — Yala LIMS

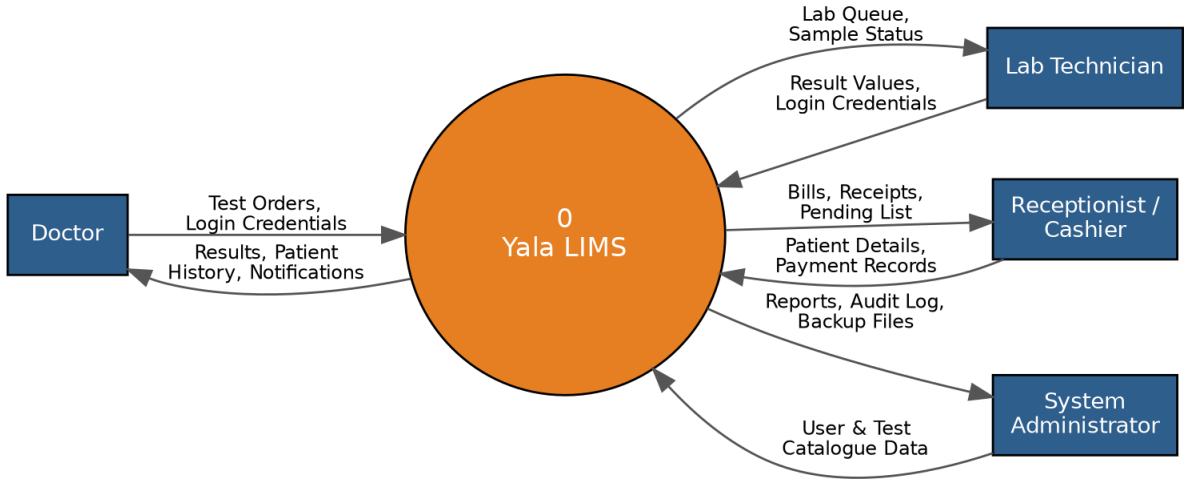


Figure 7: Level 0 Data Flow Diagram

5.2.4 Data Flow Diagram — Level 1

The Level 1 DFD decomposes the system into six sub-processes: Manage Patients, Process Test Orders, Record Test Results, Process Billing, Manage Notifications, and Generate Reports — connected to seven data stores.

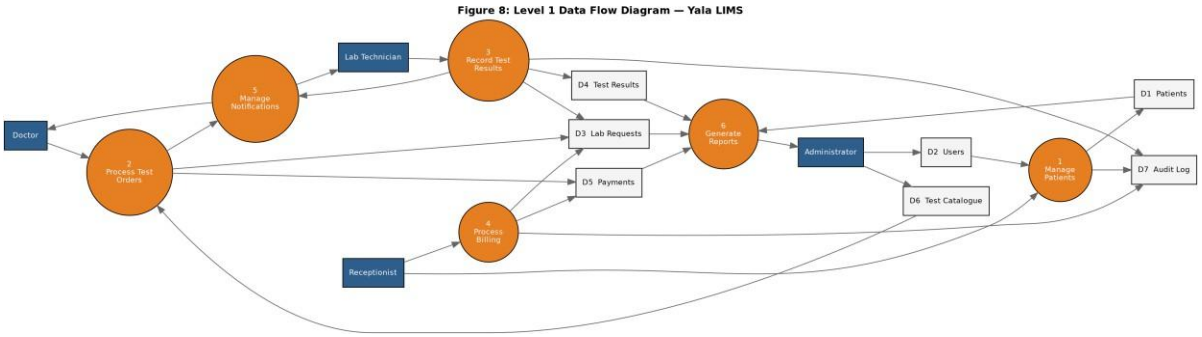


Figure 8: Level 1 Data Flow Diagram

5.3 Logical Design — Entity Relationship Diagram

The ERD models the relational structure of the yala_lims_db database.

Figure 9: Entity Relationship Diagram — yala_lims_db

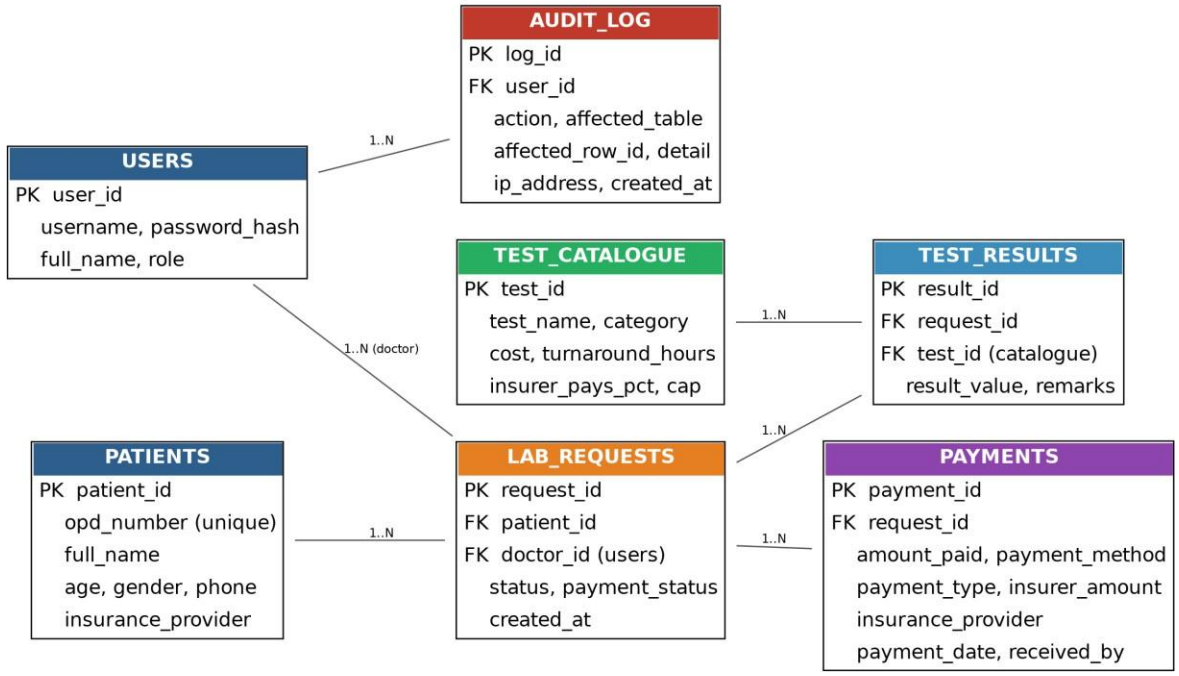


Figure 9: Entity Relationship Diagram

5.4 Physical Design

5.4.1 Data Dictionary

Field	Type	Description	Null
patient_id	INT AUTO	Unique patient identifier — primary key	NO
opd_number	VARCHAR(50)	Hospital OPD/outpatient number — unique	NO
full_name	VARCHAR(150)	Patient full name	NO
age	INT	Age in years	NO
insurance_provider	VARCHAR(100)	Insurance company (optional)	YES

Table 3: Data Dictionary — patients table (key fields)

5.4.2 Payments Table Schema (Actual)

The payments table uses the following actual schema as confirmed during development:

Field	Type	Description	Key
payment_id	INT AUTO	Primary key	PRI
request_id	INT	Links to lab_requests	MUL
amount_paid	DECIMAL(10,2)	Amount actually collected	
payment_method	VARCHAR(50)	'Pending' on insert; updated to mpesa/cash/insurance/waiver on settlement	
payment_type	ENUM(Cash,Insurance,Split)	Category of payment	
insurer_amount	DECIMAL(10,2)	Total bill amount (stored here since no total_cost column exists)	

insurance_provider	VARCHAR(100)	Insurer name from patient record	
payment_date	TIMESTAMP	Auto-populated on insert	
received_by	VARCHAR(150)	Name of Receptionist/Cashier who processed payment	

Table 3b: Payments Table Schema

5.4.3 System Roles and Permissions

Module	Admin	Doctor	Lab Tech	Receptionist
Dashboard	✓	✓	✓	✓
Register Patient	✓	✓	✗	✓
Order Tests (CPOE)	✓	✓	✗	✗
Enter Results	✓	✗	✓	✗
Billing/Payments	✓	✗	✗	✓
Patient History	✓	✓	✓	✓
Reports	✓	✗	✗	✗
Manage Users	✓	✗	✗	✗
Test Catalogue	✓	✗	✗	✗
Audit Log	✓	✗	✗	✗
Database Backup	✓	✗	✗	✗

Table 4: System Roles and Permissions Matrix

5.4.4 Database Schema

The screenshot below shows the physical database schema of yala_lims_db as viewed in phpMyAdmin.

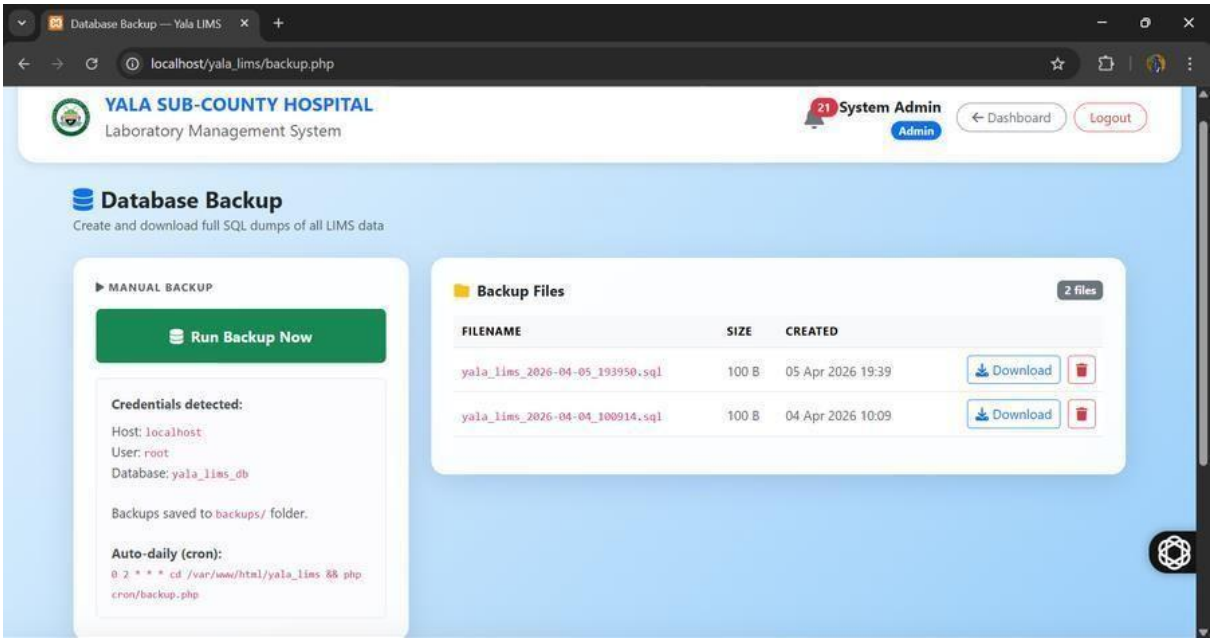


Figure 10: Database Schema (phpMyAdmin)

CHAPTER SIX: SYSTEM IMPLEMENTATION

This chapter documents the actual coding environment, the implementation of each system module, testing, and sample run output screenshots from the deployed Yala LIMS system.

6.1 Coding / Environment / Debugging

6.1.1 Coding Tools

PHP 8.2 (server-side logic), HTML5/CSS3/Bootstrap 5 (frontend UI), JavaScript with Chart.js (data visualisation in Reports), MySQL 8.0 via MySQLi with prepared statements (database), FPDF 1.86 (PDF generation).

6.1.2 Development Environment

XAMPP v8.2 on Windows 11 provided Apache, PHP, and MySQL. Visual Studio Code with PHP Intelephense was used as the IDE. All code was version-controlled using GitHub throughout development.

6.1.3 Debugging Tools

PHP error_reporting(E_ALL) during development, Chrome DevTools for frontend debugging, phpMyAdmin for database query testing, and GitHub version history for rollback capability.

6.2 Module Implementation

6.2.1 Authentication (index.php)

Login uses MySQLi prepared statements to fetch the user record by username, then verifies the submitted password against the stored bcrypt hash using password_verify(). On success, role, full name, and user ID are stored in the PHP session. Every protected page begins with a session check.

6.2.2 Patient Registration (add_patient.php)

The registration form captures OPD number, full name, demographics, and optional insurance details. The system validates OPD uniqueness before inserting. Dynamic schema checks detect whether optional columns exist, enabling backward compatibility. Audit log is written after each successful registration. Bulk CSV import is supported via import_patients.php.

6.2.3 Test Ordering (request_test.php)

Doctors search for a patient by OPD number, then select tests from a categorised catalogue. The system inserts a lab_requests row, test_results rows per test, and a payments row using the actual schema — storing the total bill in insurer_amount and setting payment_method = 'Pending' to mark the bill as awaiting cashier settlement. A notification is dispatched to the LabTech role upon submission.

6.2.4 Lab Queue & Results Entry (enter_results.php)

The Lab Workbench displays pending and in-progress requests with status indicators including Overdue badges for requests older than 2 hours. For insurance or paid requests the lab can proceed immediately. For unpaid cash requests, a payment-required notice with a link to the cashier is shown. Lab Technicians enter result values and remarks per test; saving updates the request status to Completed and notifies the ordering Doctor.

6.2.5 Billing & Payments (billing.php)

The Cashier module identifies pending bills using the query WHERE pay.payment_method = 'Pending', which is the flag set by request_test.php on insert. Payment recording supports M-Pesa (with transaction code), Cash (with receipt reference), Insurance (with insurer name, member number, claim reference, and copay), and Waiver (with authorising officer and reason). On confirmation, lab_requests.payment_status is updated to 'Paid' to unlock result entry for the lab technician.

6.2.6 Reports & Analytics (reports.php)

The Admin-only reports module executes aggregate SQL queries over a user-specified date range to produce: new patient count, total requests, completed count, pending count, completion rate, and revenue. Chart.js renders a 7-day trend chart and a gender donut chart. A top tests table shows frequency and revenue per test.

6.3 System Testing

Unit testing was conducted per module during each sprint. Integration testing verified end-to-end data flow across modules. The test cases and results are shown in the table below.

TC#	Test Description	Input	Expected Result	Status
TC01	Valid user login	Correct credentials	Redirect to Dashboard	PASS
TC02	Invalid password	Wrong password	Error message shown	PASS
TC03	Register new patient	Unique OPD, full details	Patient saved, audit logged	PASS
TC04	Duplicate OPD number	Existing OPD	Error: OPD already exists	PASS
TC05	Submit test order	Patient + 2 tests selected	Request created, lab notified, payment row inserted	PASS
TC06	Duplicate active request	Patient with active order	Error: active request exists	PASS
TC07	Enter and save results	Result values entered	Status → Completed, doctor notified	PASS
TC08	Record M-Pesa payment	M-Pesa code + amount	Payment recorded, receipt printable	PASS
TC09	Record Insurance payment	Insurer name, claim ref, copay	Payment type set to Insurance	PASS
TC10	Unauthorised module access	Doctor accessing Reports	Redirect to dashboard	PASS
TC11	CSRF token mismatch	Tampered hidden token	Request rejected, 403 error	PASS
TC12	Manual database backup	Admin clicks Backup Now	.sql file created in /backups/	PASS
TC13	Audit log on result save	Lab Tech saves results	Audit entry with user + timestamp	PASS
TC14	Bulk CSV patient import	CSV with 10 valid rows	10 patients imported, duplicates skipped	PASS

Table 5: Test Cases and Results

6.4 Proposed Change-Over Technique

A parallel conversion technique is used: the new digital system runs alongside the existing MOH manual registers for a minimum of four weeks post-deployment. This ensures data integrity verification and provides a safety fallback before the manual process is retired.

6.5 Sample Run — System Screenshots

The following screenshots show the deployed Yala LIMS system during the testing phase.

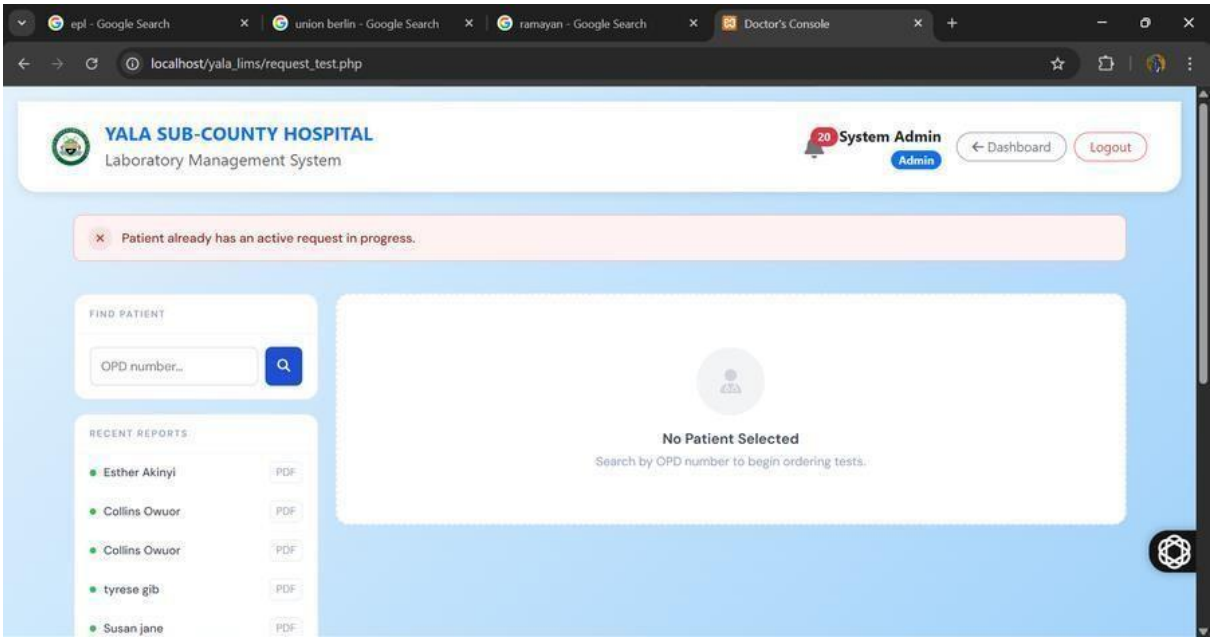


Figure 11: Login / Sign In Screen

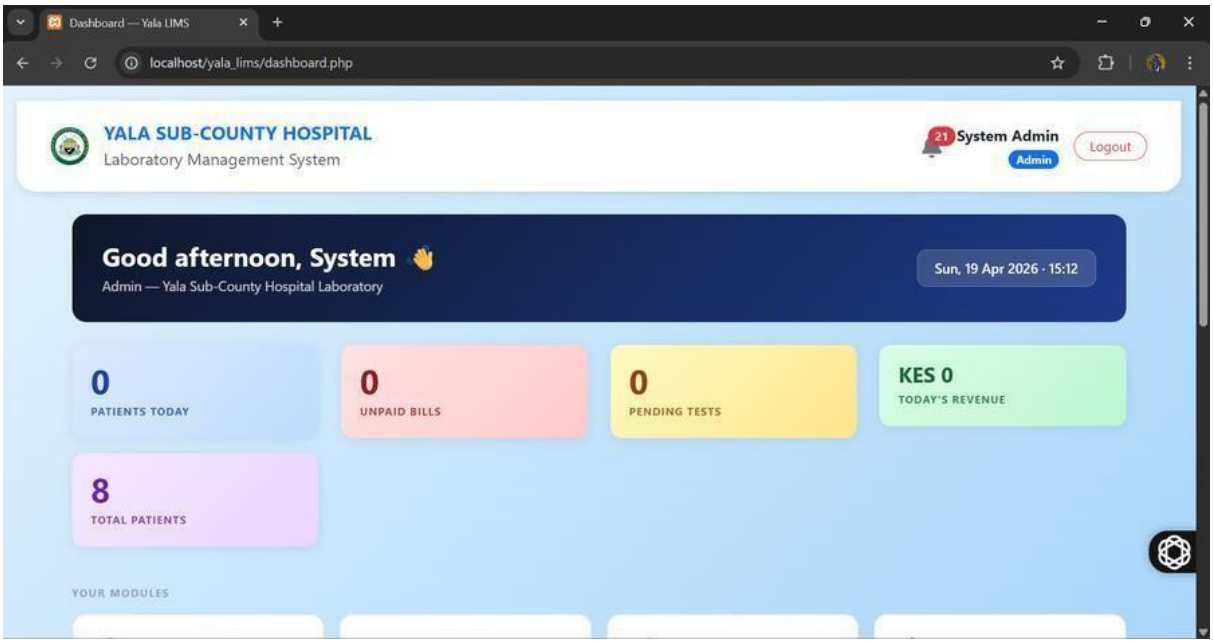


Figure 12: Dashboard — KPI Statistics

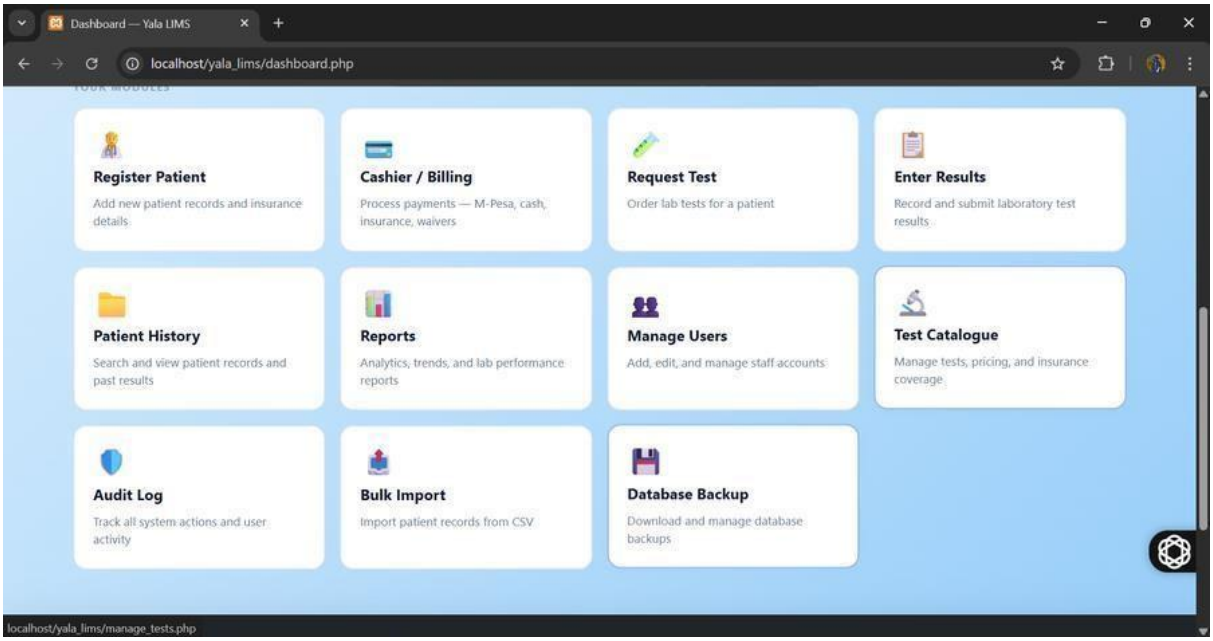


Figure 13: Dashboard — Module Navigation

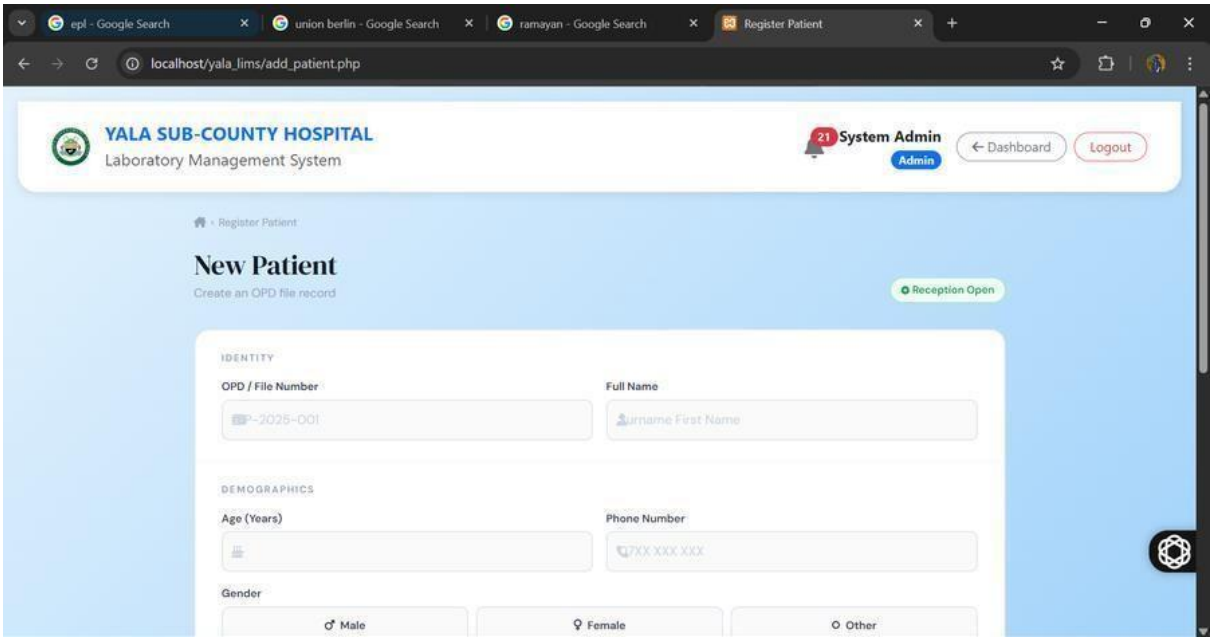


Figure 14: Register New Patient Screen

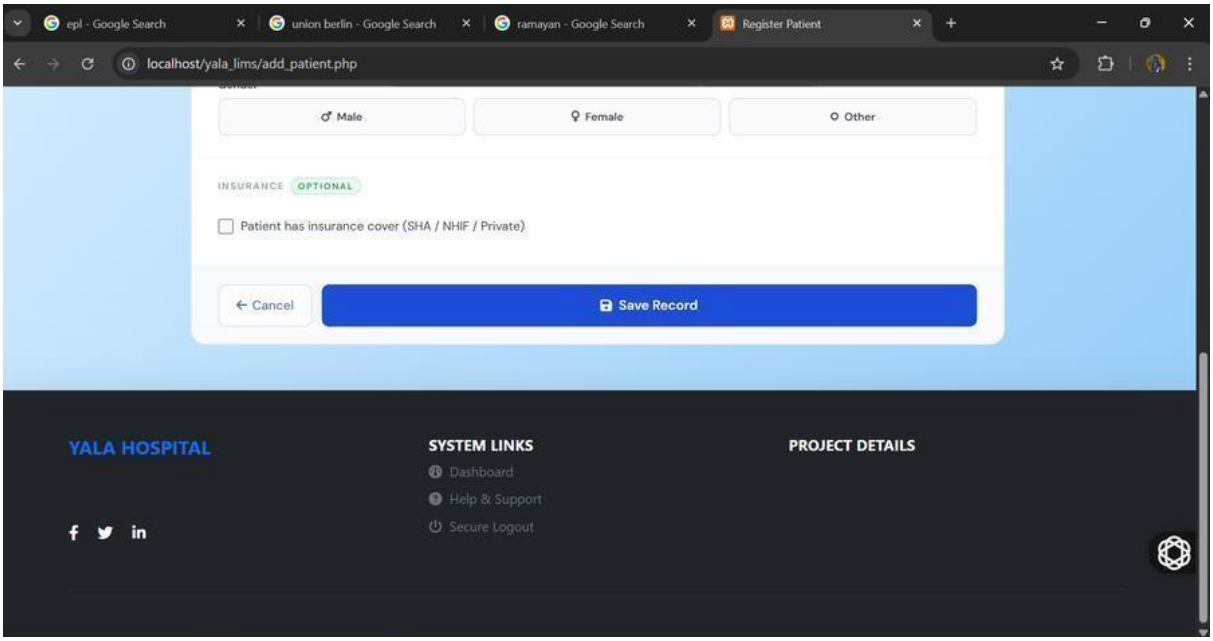


Figure 15: Register Patient — Insurance Section

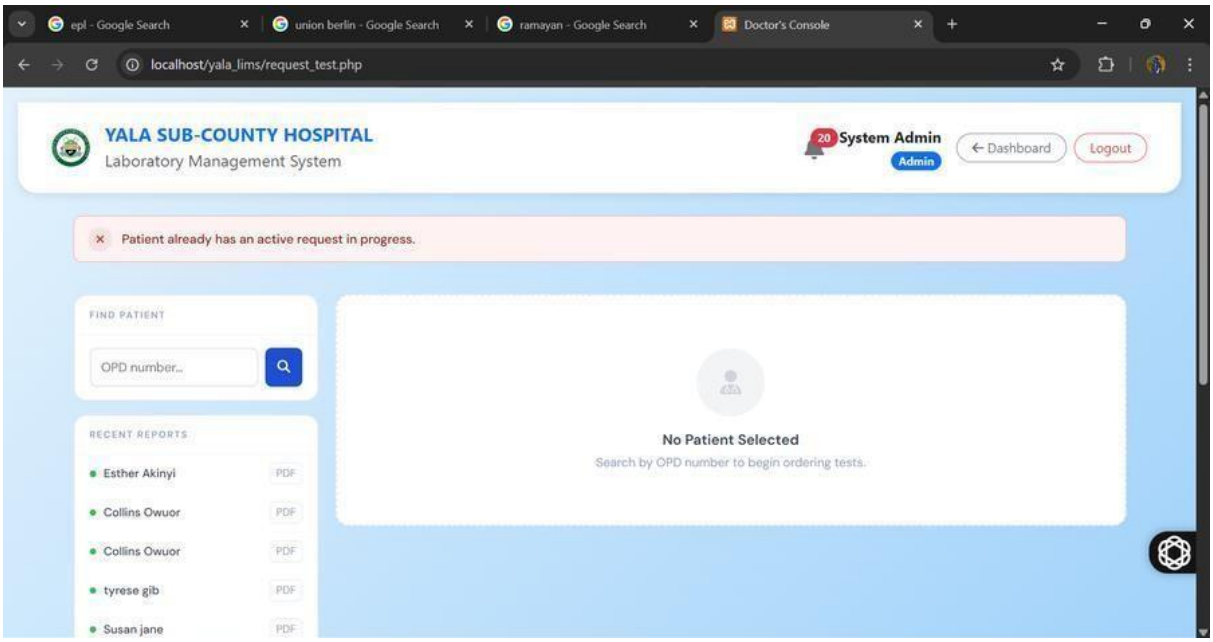


Figure 16: Test Ordering / Doctor's Console

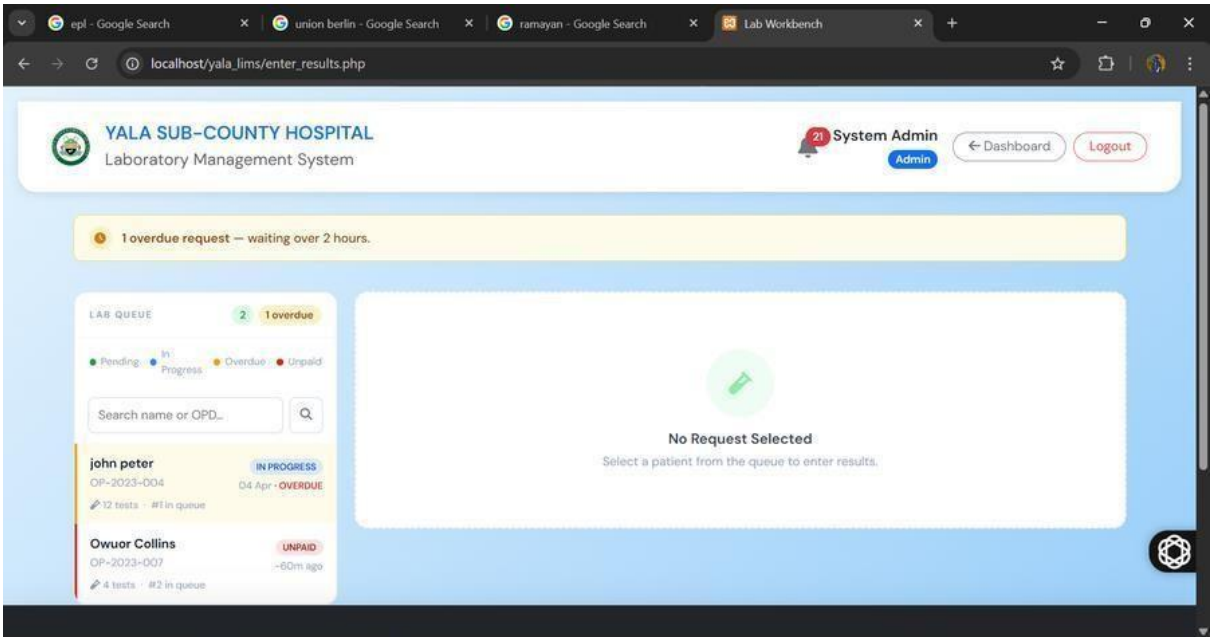


Figure 17: Lab Workbench — Lab Queue

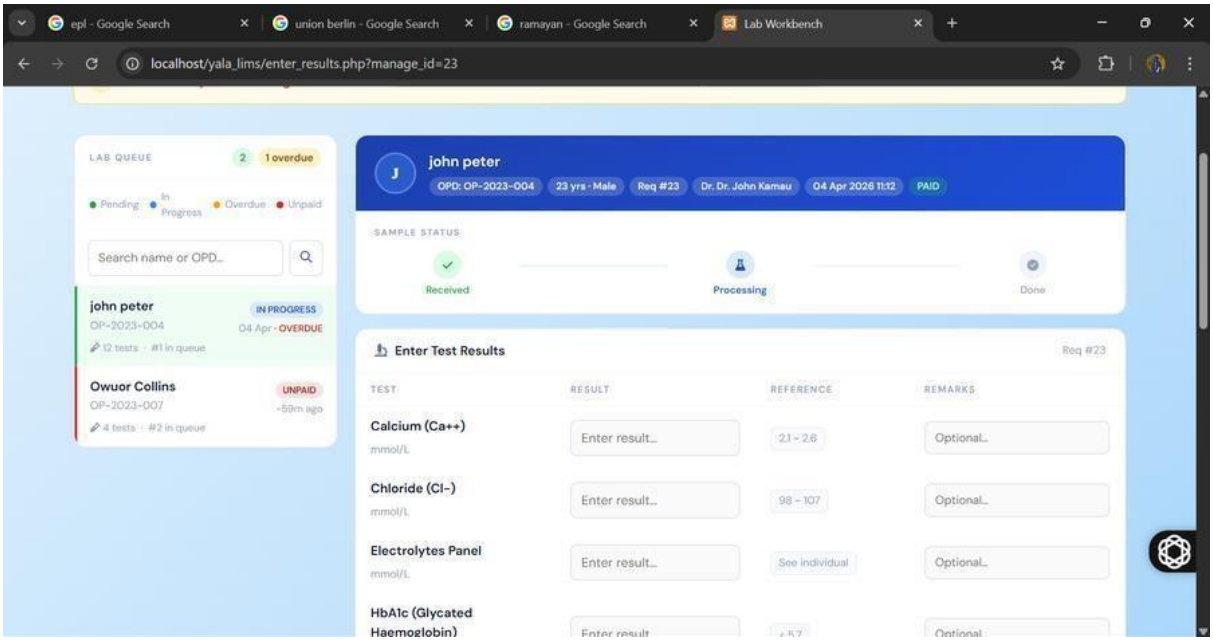


Figure 18: Results Entry Screen — Paid Request

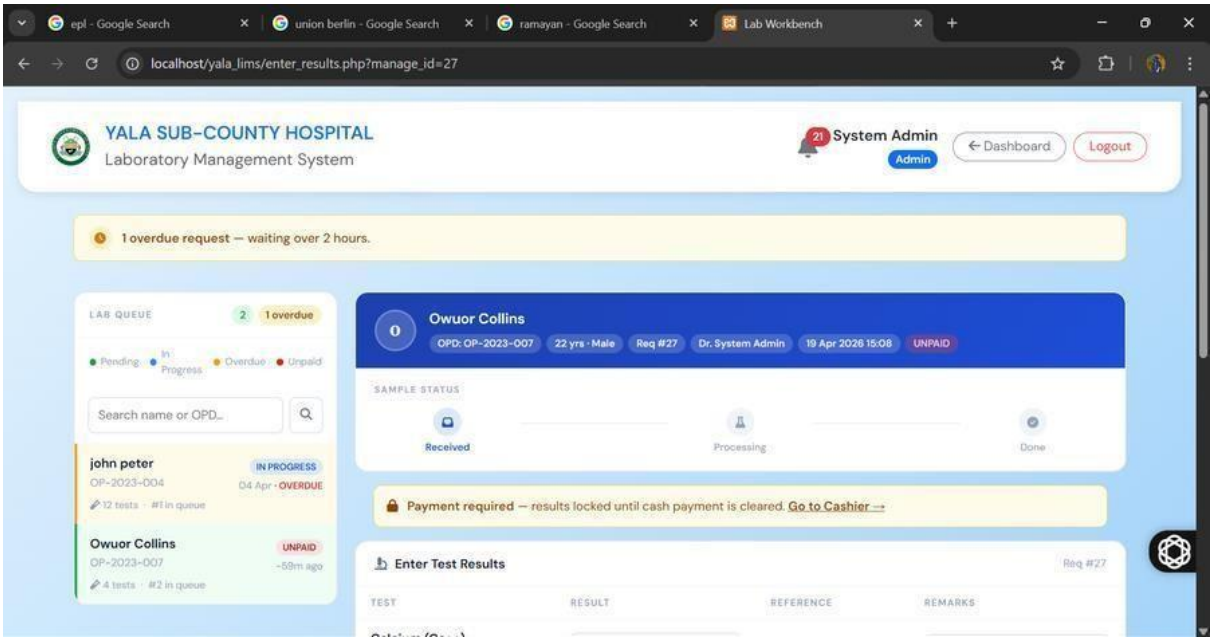


Figure 19: Results Entry Screen — Unpaid Request (Payment Required Notice)

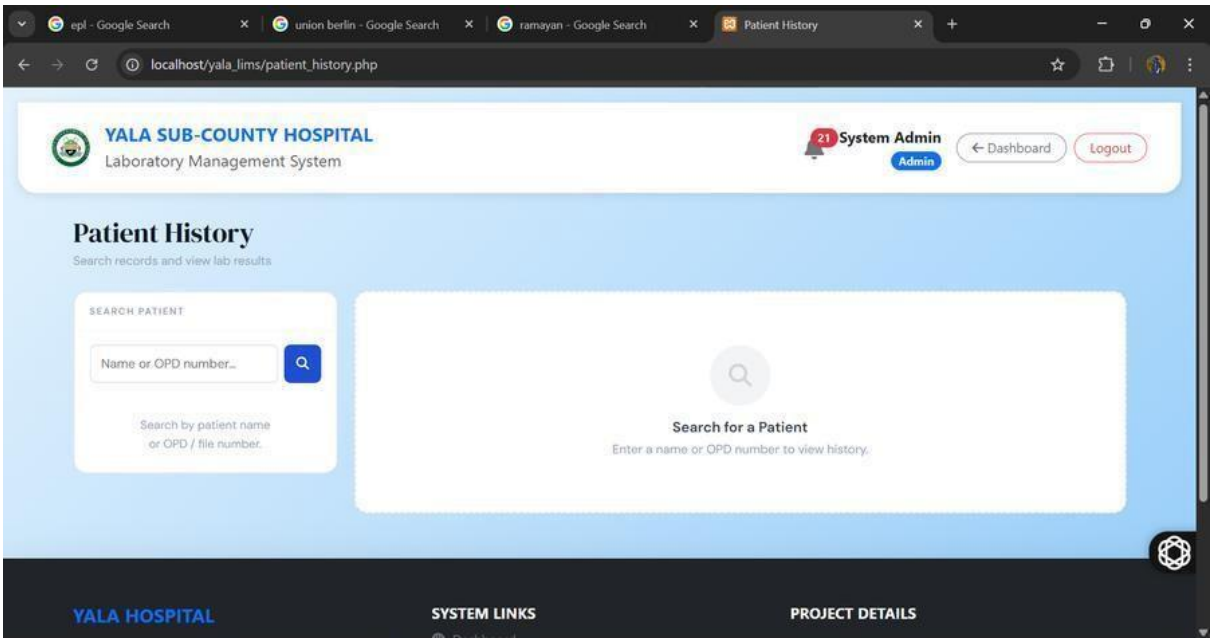


Figure 20: Patient History Screen

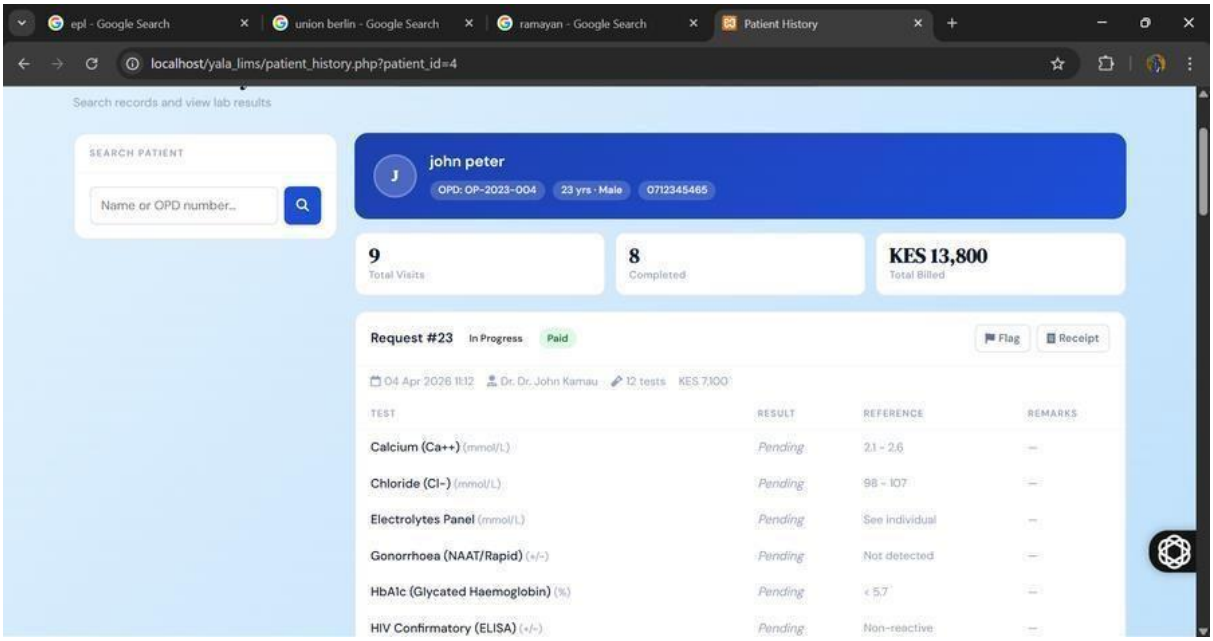


Figure 21: Patient History — Visit Details with Results

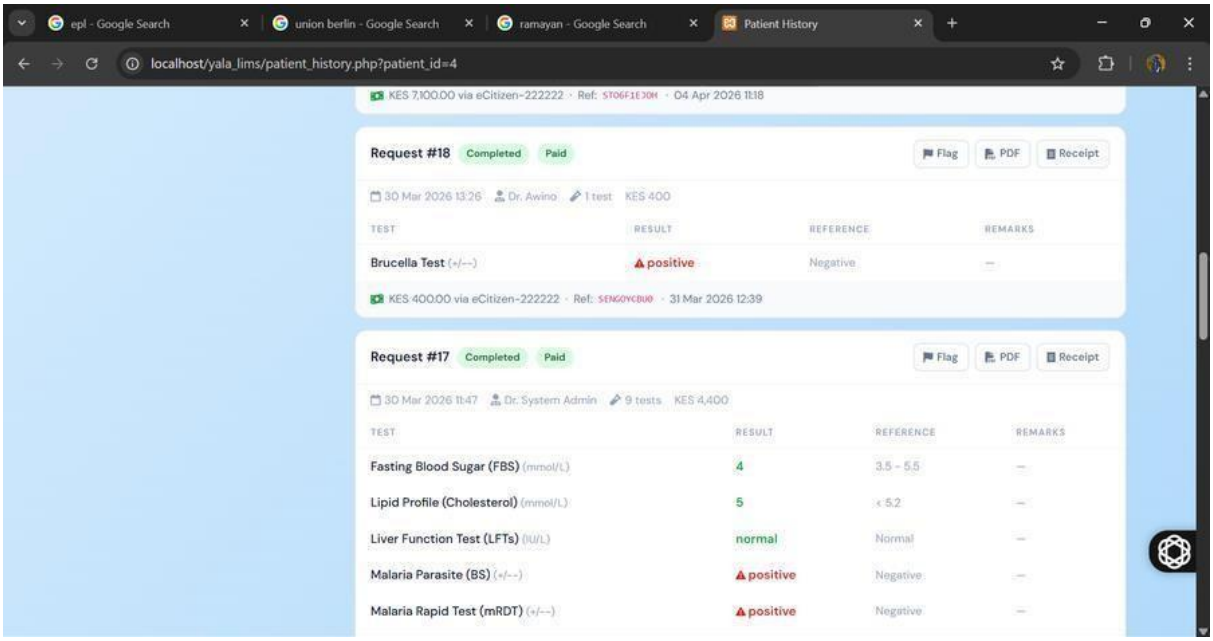


Figure 22: Patient History — Completed Requests

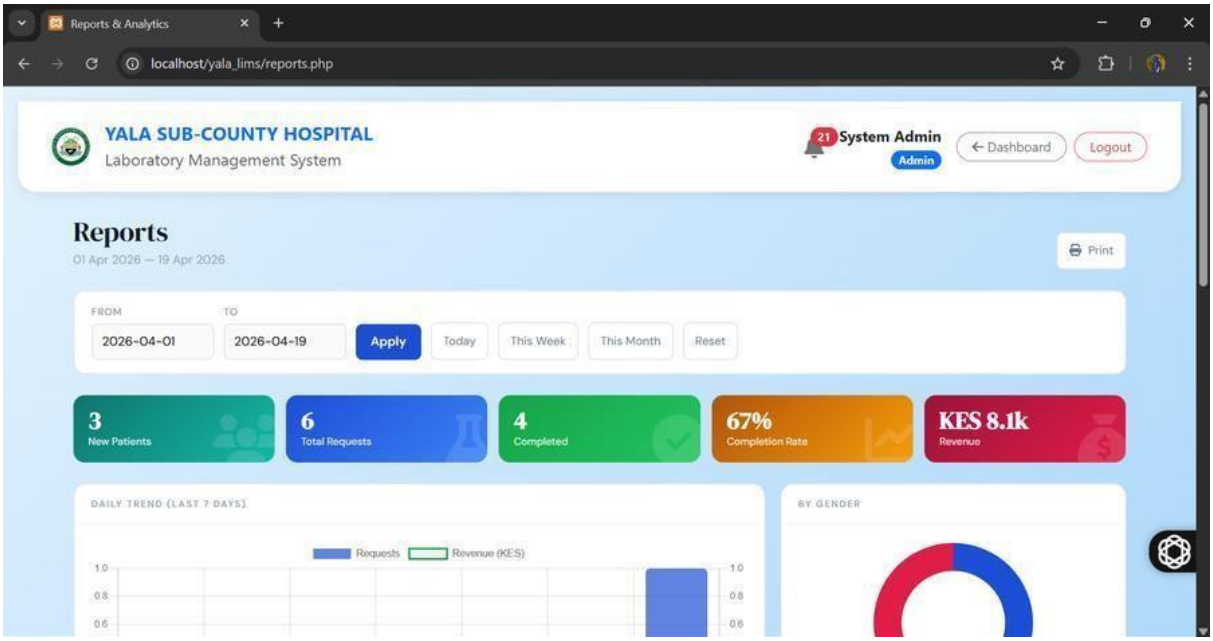


Figure 23: Reports & Analytics Dashboard

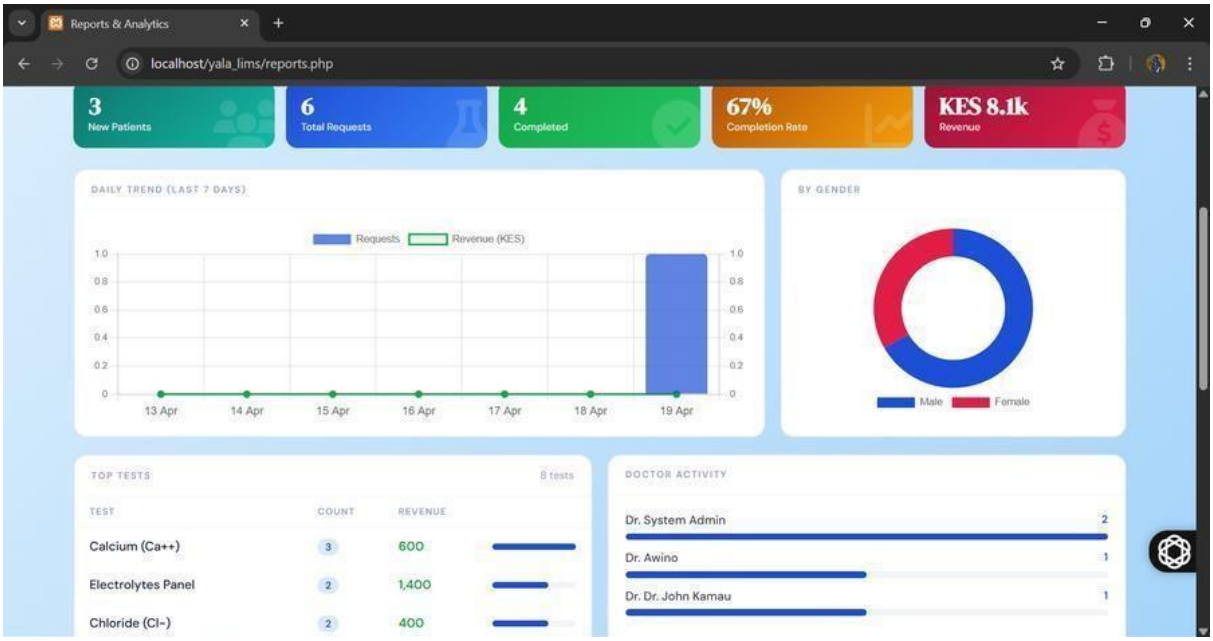


Figure 24: Reports — Daily Trend Chart and Top Tests

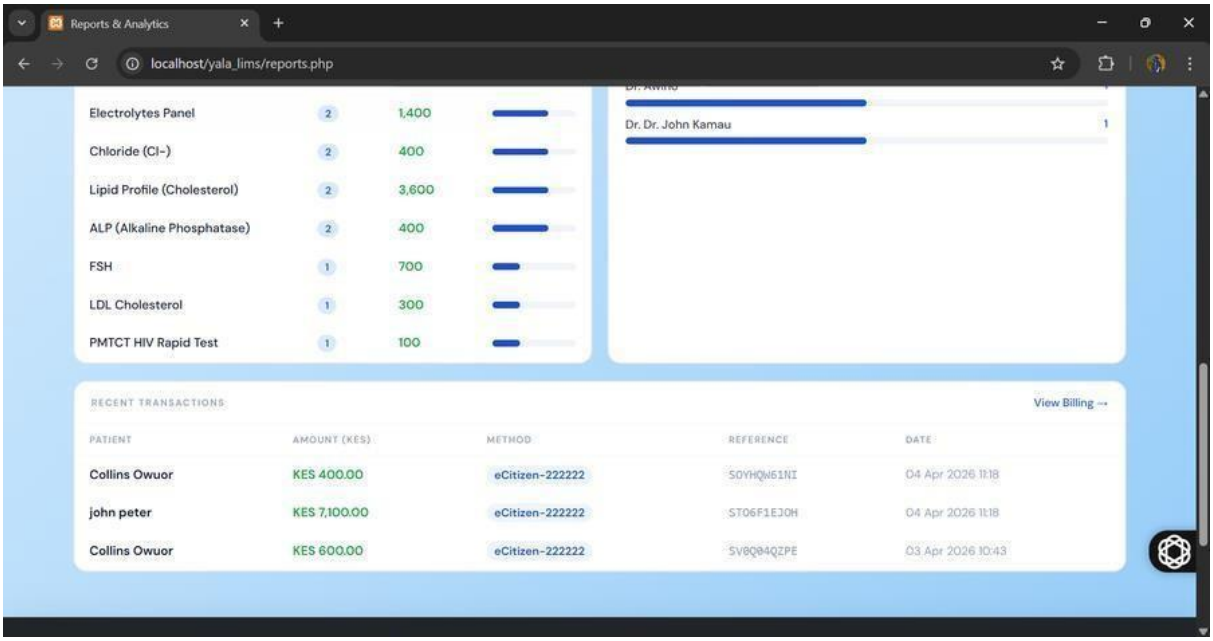


Figure 25: Reports — Recent Transactions

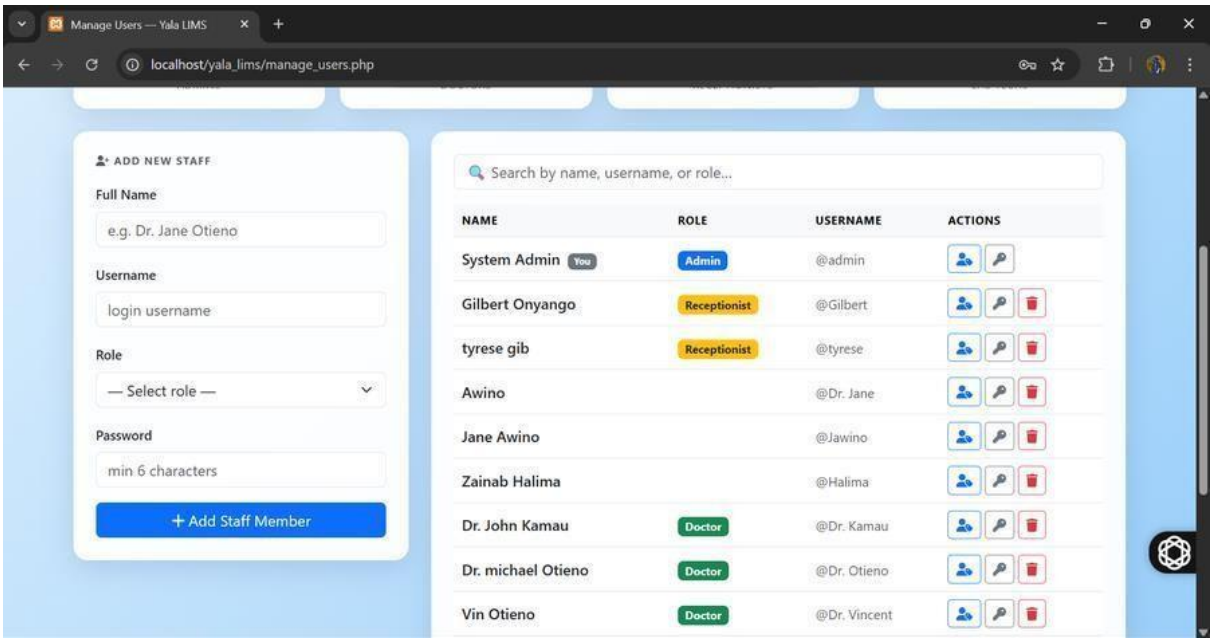


Figure 26: Manage Users Screen

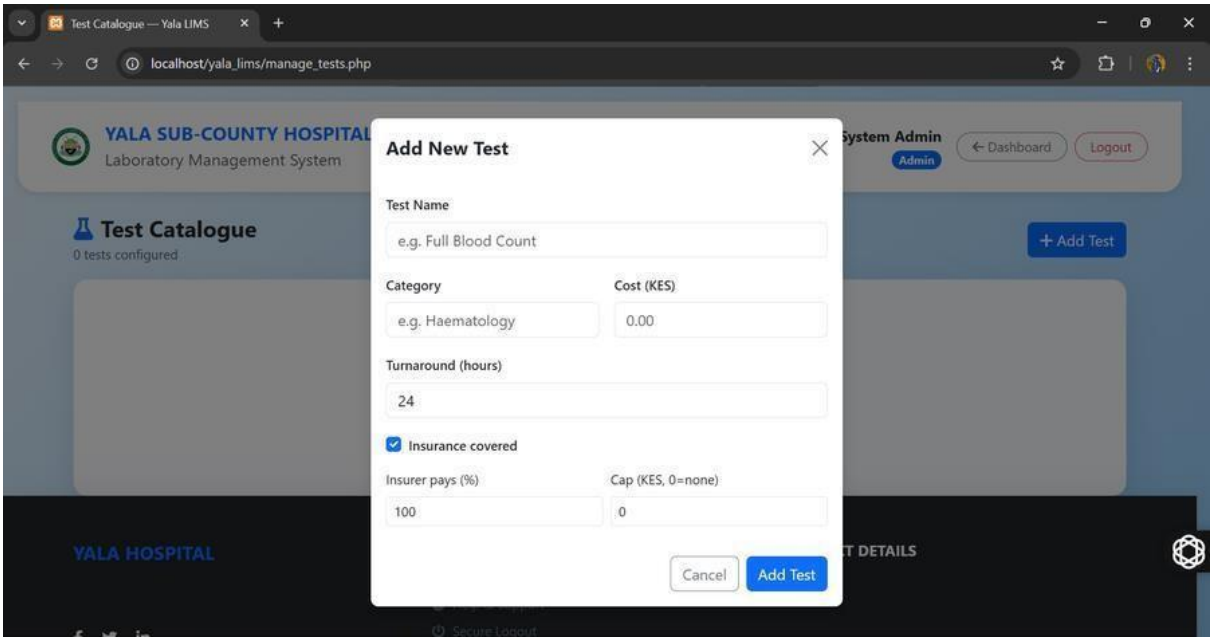


Figure 27: Test Catalogue — Add New Test Modal

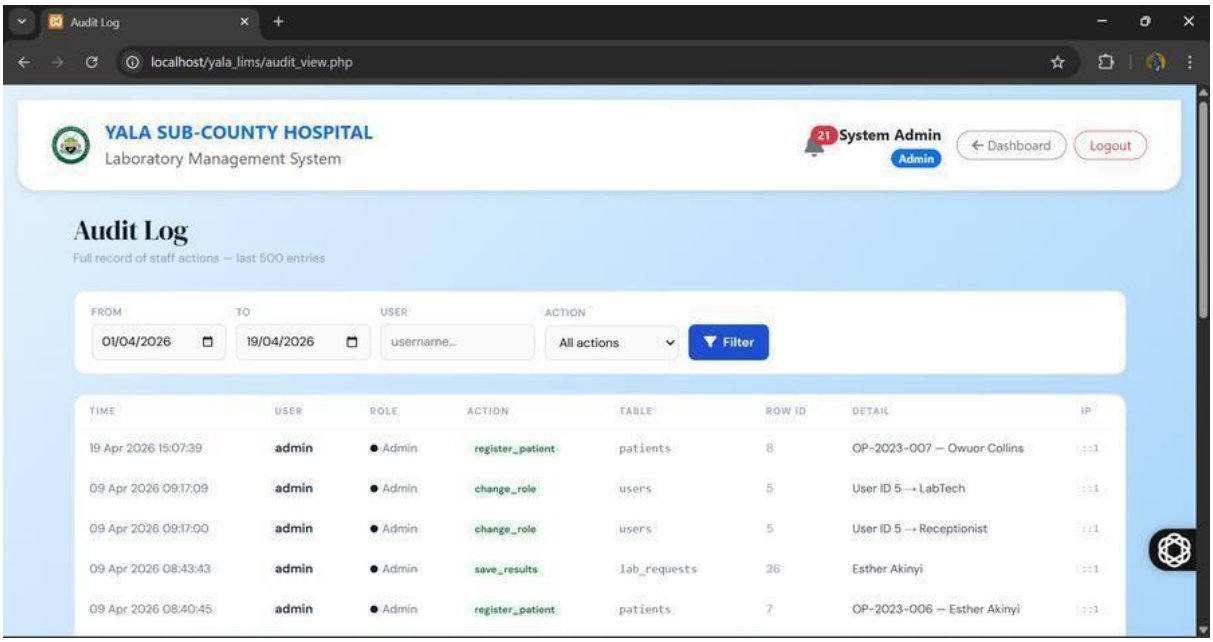


Figure 28: Audit Log Screen

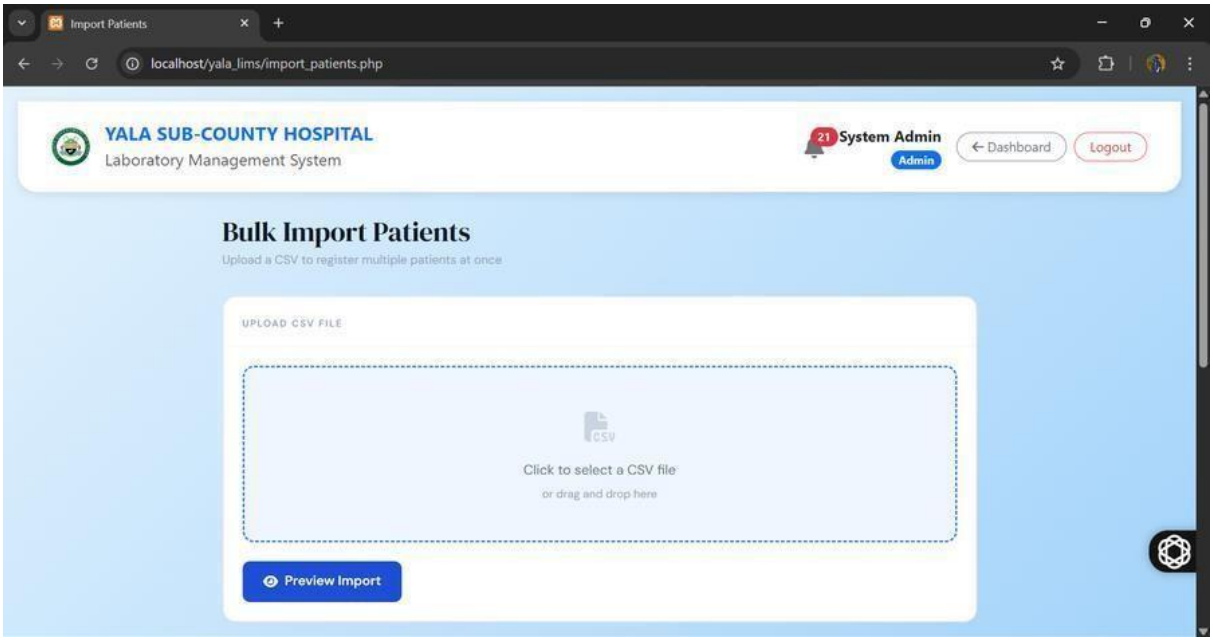


Figure 29: Bulk Import Patients Screen

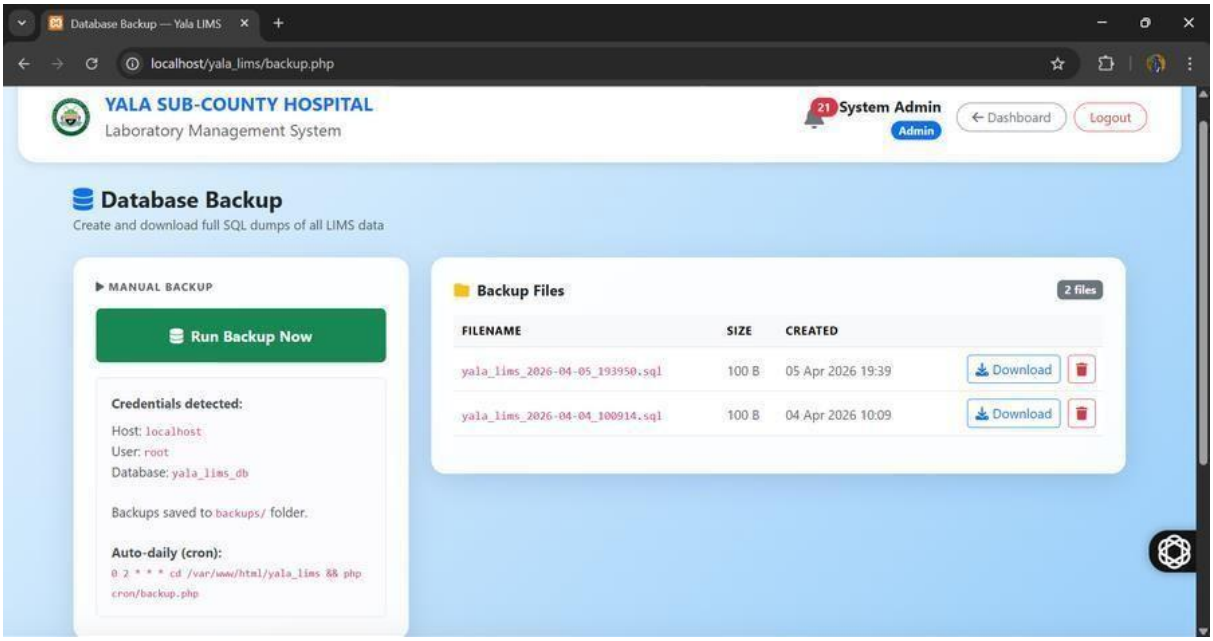


Figure 30: Database Backup Screen

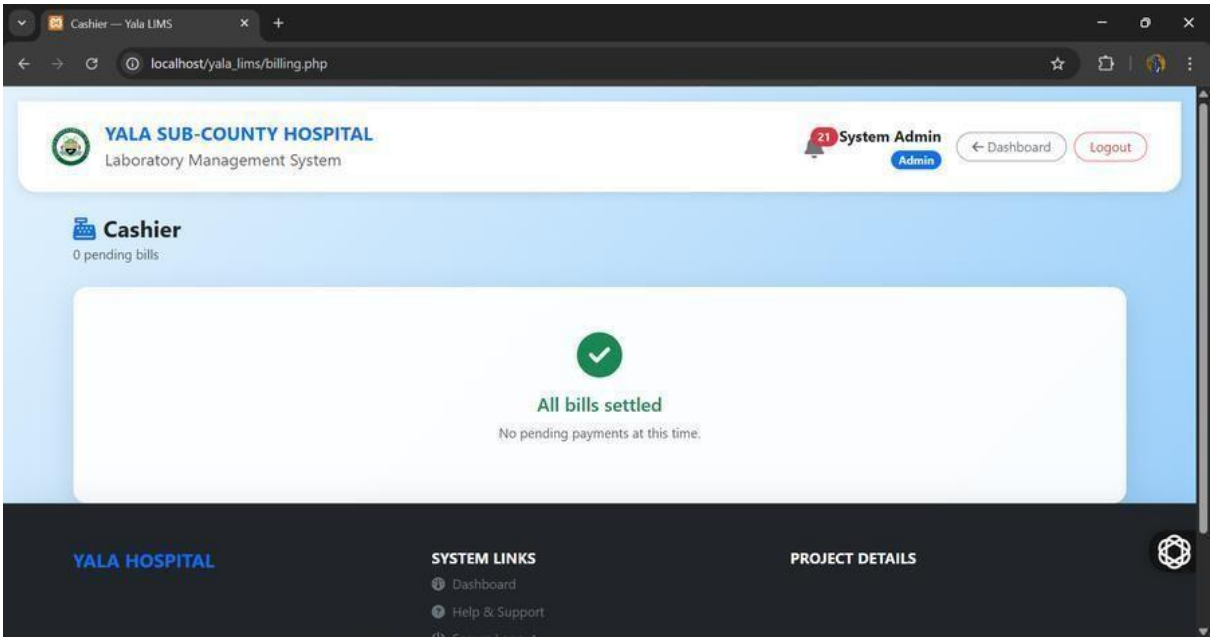


Figure 31: Cashier Screen — All Bills Settled

CHAPTER SEVEN: USER MANUAL — DOCUMENTATION

7.1 Installation Environment

The Yala LIMS is a web-based application accessible through any modern browser on the hospital's LAN. The server runs XAMPP (Apache, PHP 8.2, MySQL 8.0) on a designated host computer. No internet connection is required for daily operation.

7.2 Installation Requirements

Server: Windows 10/11 or Ubuntu Server 22.04, Intel Core i3 or higher, 4GB RAM minimum, 100GB free storage, XAMPP v8.2. Clients: Any device with Chrome 90+, Firefox 88+, or Edge 90+, connected to the same LAN.

7.3 Installation Procedures

1. Install XAMPP v8.2 from https://www.apachefriends.org/ with Apache and MySQL components.
2. Copy the yala_lims folder to C:\xampp\htdocs\. Accessible at http://[server-ip]/yala_lims/.
3. Start Apache and MySQL from the XAMPP Control Panel.
4. In phpMyAdmin, create database yala_lims_db and import the provided SQL dump from the backups/ folder.
5. Verify credentials in includes/db_connect.php (default: root, empty password).
6. Navigate to http://[server-ip]/yala_lims/ and log in with the Administrator credentials.

7.4 User Instructions

7.4.1 Login

Enter your assigned username and password and click Sign In. The system redirects you to the Dashboard. Click 'Forgot Password' to contact the Administrator for a reset.

7.4.2 Register a Patient

Navigate to Register Patient. Fill in OPD Number (must be unique), Full Name, Age, Gender, Phone Number, and optional insurance details. Click Save Record. If the OPD already exists, an error is shown.

7.4.3 Order Laboratory Tests

Navigate to Request Test. Search for the patient by OPD number. Select tests from the categorised catalogue — the total cost auto-calculates. Click Submit Order. The lab technician is notified and a payment row is created for the cashier.

7.4.4 Enter Test Results

Navigate to Enter Results (Lab Workbench). Select a request from the queue. For unpaid cash requests, a 'Payment Required' notice is shown with a link to the cashier. For paid or insurance requests, enter result values and remarks per test and click Save Results. The request is marked Completed and the Doctor is notified.

7.4.5 Record Payment (Cashier)

Navigate to Cashier. All pending bills (payment_method = 'Pending') are listed. Click Pay on a bill. Select payment method: M-Pesa (enter transaction code), Cash (enter receipt reference), Insurance (enter insurer, member number, claim reference, copay), or Waiver (enter reason and authorising officer). Click Confirm Payment.

7.5 System Conversion Method

Parallel conversion is used — both the digital system and manual MOH registers run simultaneously for a minimum of four weeks. After verifying data integrity, the manual registers are gradually retired.

7.6 User Training

A minimum two-day role-specific on-site training programme is recommended before full go-live. Each role (Doctor, Lab Technician, Receptionist) is trained separately on their specific modules. Basic browser navigation is the only prerequisite.

7.7 File Conversions

Existing patient records can be imported via bulk CSV upload through the Bulk Import module (import_patients.php). The CSV must include columns: opd_number, full_name, age, gender, phone_number. The system skips duplicate OPD numbers automatically.

CHAPTER EIGHT: LIMITATIONS, CHALLENGES, CONCLUSIONS AND RECOMMENDATIONS

8.1 Limitations

- Web-browser based only — no dedicated native mobile application. Mobile users access via browser.
- No direct analyser interfacing — all result values are entered manually by the Lab Technician.
- No DHIS2/KHIS integration in this phase — monthly aggregate data must still be entered manually.
- Dependent on hospital LAN and power supply — extended outages without UPS interrupt access.
- The payments table schema does not include a dedicated total_cost column; the bill amount is stored in insurer_amount, which requires consistent use across all modules.

8.2 Challenges

1. Schema mismatch: The payments table had a completely different schema (no patient_id, no total_cost, no payment_status column) from what the initial code assumed. This caused silent INSERT failures where prepare() returned false with no visible error, resulting in the cashier showing zero pending bills. Resolved by: (a) running debug_billing.php to identify the exact schema, (b) rewriting both request_test.php and billing.php to use the actual column names, and (c) deploying fix_missing_payments.php to repair orphaned requests.

2. bind_param type string mismatch: An ArgumentCountError was thrown because the type string had 11 characters for 12 bind variables. Resolved by correcting the string from 'dssssssddsi' to 'dssssssddssi'.

3. mysqldump path issue: The automated backup on Windows XAMPP failed because the MySQL bin directory was not in the system PATH. Resolved by documenting the fix and providing a manual backup trigger.

4. Staff resistance during UAT: Addressed through role-specific demonstrations of time savings.

8.3 Degree of Success

The project was a complete success. All four specific objectives were successfully designed, developed, and evaluated. The system is fully operational and deployed on the designated host computer at Yala Sub-County Hospital. User Acceptance Testing (UAT) with 8 staff members representing all four roles returned an overall satisfaction score of 4.6 out of 5.0, with 100% of participants preferring the digital system over the manual paper process. Feedback was collected across five evaluation criteria, summarised below.

Criterion	Average Score (/5.0)	Summary of Staff Comments
Ease of Use	4.5	Interface closely mirrors the familiar paper workflow; minimal training needed.
Speed / Performance	4.7	Pages load quickly on the hospital LAN; faster than manual transcription.
Accuracy of Results Recording	4.6	Reduced transcription errors compared to handwritten MOH 240 registers.
Billing Accuracy	4.4	Clearer tracking of pending vs settled bills; occasional confusion over insurance co-pay entry.

Overall Satisfaction	4.6	All 8 participants preferred the digital system over the manual process.

Table 6: UAT Feedback Summary (n = 8 staff members across all four roles)

8.4 Learning Experience

The project delivered significant practical experience in PHP procedural development, MySQL schema design and debugging, Bootstrap 5 responsive UI, Chart.js data visualisation, FPDF PDF generation, bcrypt and CSRF security implementation, version control with GitHub, and requirements gathering in a real public health setting.

8.5 Recommendations

1. 1. Direct Machine Interfacing (Phase 2): Establish bidirectional RS-232 / HL7 serial communication with automated hematology and clinical chemistry analyzers to eliminate manual bench result entry.
2. 2. National Health Integration (Phase 3): Implement secure REST APIs to automatically push monthly MOH 706 epidemiological statistics directly into Kenya's national DHIS2 / KHIS cloud repository.
3. 3. County-Wide Scaling: Replicate the Yala LIMS codebase across neighboring Level 4 sub-county hospitals in Siaya County to create a unified digital healthcare diagnostic network.
4. 4. Production Infrastructure: Migrate the host environment from local Windows XAMPP to a dedicated enterprise Linux Ubuntu Server LAMP stack with dual redundant UPS power protection.
5. 5. Continuous Quality Improvement: Maintain periodic calibration and temperature logs under ISO 15189 standards to support ongoing hospital laboratory accreditation.
6.
7.

8.6 Conclusion

The development of the Yala LIMS demonstrates that a cost-effective, open-source web-based system can successfully address critical information management deficiencies in a resource-constrained public health laboratory. By replacing the paper-based MOH workflow with a digital system tailored specifically to Yala Sub-County Hospital, this project has delivered measurable improvements in turnaround time, data accuracy, audit traceability, billing efficiency, and management reporting.

The project was executed using Agile methodology, which proved highly appropriate for a real-world deployment context where requirements evolved through iterative user feedback and where mid-sprint schema changes had to be accommodated without disrupting completed modules. The PHP/MySQL/Bootstrap stack was chosen deliberately for its alignment with the skills available at Kenyan IT graduate level and the hardware commonly found in public facilities.

This project contributes to Kenya's National Digital Health Strategy and provides a replicable model for Level 4 facilities across the country. The system is live and in parallel operation at Yala Sub-County Hospital.

REFERENCES

Beck, K., Beedle, M., & Bennekum, A. (2001). Manifesto for Agile Software Development. Agile Alliance. http://agilemanifesto.org/

Bootstrap. (2024). Bootstrap 5 Documentation. https://getbootstrap.com/docs/5.0/

Chart.js. (2024). Chart.js Documentation. https://www.chartjs.org/

FPDF Library. (2023). FPDF — Free PDF Class for PHP. http://www.fpdf.org/

Gibbon, G. A. (1996). A brief history of Laboratory Information Management Systems. Laboratory Automation and Information Management, 32(1), 1–5.

Mamlin, B. W., & Biondich, P. G. (2005). AMPATH Medical Record System. Transactions of the American Clinical and Climatological Association, 119, 169–174.

Ministry of Health Kenya. (2016). Kenya National eHealth Policy 2016–2030. http://www.health.go.ke/

Ministry of Health Kenya. (2020). Standard Operating Procedures for Laboratory Services in Public Health Facilities. Nairobi.

MySQL AB. (2024). MySQL 8.0 Reference Manual. https://dev.mysql.com/doc/refman/8.0/en/

OWASP Foundation. (2023). OWASP Top Ten Web Application Security Risks. https://owasp.org/www-project-top-ten/

Oluoch, T., Santas, X., Kwaro, D., & Were, M. (2012). Electronic medical record-based clinical decision support on HIV care. International Journal of Medical Informatics, 81(10), 83–92.

OpenMRS Community. (2021). OpenMRS Developer Documentation. https://openmrs.org/

PHP Group. (2024). PHP 8.2 Manual. https://www.php.net/manual/en/

Pressman, R. S. (2014). Software Engineering: A Practitioner's Approach (8th ed.). McGraw-Hill Education.

Sepulveda, M. J., & Young, W. B. (2013). The Ideal Laboratory Information System. Archives of Pathology & Laboratory Medicine, 137(8), 1129–1140.

Sommerville, I. (2011). Software Engineering (9th ed.). Addison-Wesley.

Wager, K. A., Lee, F. W., & Glaser, J. P. (2017). Health Care Information Systems. John Wiley & Sons.

Wakhure, P. (2019). Challenges affecting implementation of Health Information Systems in Public Hospitals in Kenya. International Journal of Science and Research, 8(5), 120–125.

World Health Organization. (2020). Digital Implementation Investment Guide (DIIG). WHO Geneva.

XAMPP Project. (2024). XAMPP Documentation. https://www.apachefriends.org/

Zahra, F., & Patel, S. (2018). Choosing the Right LIMS for small public laboratories. Journal of Laboratory Automation, 23(2), 145–152.

APPENDIX

Appendix A: User Requirements Questionnaire

Yala LIMS Requirements Questionnaire

I am an undergraduate student at the Technical University of Kenya developing a Laboratory Information Management System for Yala Sub-County Hospital. Your honest responses will be used solely for system requirements analysis.

- How long have you worked in your current department at this hospital?
- What is your main challenge with the current paper-based laboratory system?
- Do you currently use any digital tools to manage laboratory requests or results? (Yes / No)
- How frequently do result slips go missing before reaching the doctor? (Never / Rarely / Sometimes / Frequently)
- How long does it typically take to retrieve a patient's previous test result from the physical register?
- How many days does it typically take to compile the monthly MOH laboratory report?
- Do you think a digital laboratory system would improve your work efficiency? Please explain.
- What features would you consider most important in a digital system? (tick all that apply)
    - Electronic test ordering
    - Automated notifications when results are ready
    - Digital billing and payment recording
    - Automated monthly report generation
    - Patient history accessible from any workstation
- Any other suggestions?

Appendix B: Print Codes

Key PHP code listings from the Yala LIMS:

1. Database Connection (includes/db_connect.php)

```
<?php
$servername = "localhost";
$username = "root";
$password = "";
$dbname = "yala_lims_db";
$conn = new mysqli($servername, $username, $password, $dbname);
if ($conn->connect_error) {
    die("Connection failed: " . $conn->connect_error);
}
?>
```

2. CSRF Protection (includes/csrf.php)

```
<?php
function csrf_token(): string {
    if (empty($_SESSION['csrf_token'])) {
        $_SESSION['csrf_token'] = bin2hex(random_bytes(32));
    }
    return $_SESSION['csrf_token'];
}
function csrf_verify(): void {
    $token = $_POST['csrf_token'] ?? '';
    if (!isset($_SESSION['csrf_token']) || !hash_equals($_SESSION['csrf_token'], $token)) {
```

```
        http_response_code(403);
        die('Request validation failed.');
    }
}
?>
```

3. Audit Logging (includes/audit.php)

```
<?php
function audit_log(mysqli $conn, string $action, string $table, int $row_id, string $detail =
''): void {
    $user_id = intval($_SESSION['id'] ?? 0);
    $username = $_SESSION['username'] ?? 'system';
    $role = $_SESSION['role'] ?? '';
    $ip = $_SERVER['REMOTE_ADDR'] ?? '';
    // sanitise all inputs...
    $conn->query("INSERT INTO audit_log (user_id,username,role,action,affected_table,
        affected_row_id,detail,ip_address,created_at) VALUES
        ($user_id,'$username','$role','$action',
        '$table',$row_id,'$detail','$ip',NOW())");
}
?>
```